NextHire

Software Requirements Specification (SRS)

An AI-Powered Resume Filtering and Recruitment Platform

Prepared by: Team Innovex

SL	Name	Student ID	Role
01	Afsana	230102053	Team Leader
02	Sabiha Akter	230102051	Database Administrator
03	Tasnim Ahmed Arnob	230102042	Backend Engineer
04	Mahbubur Rahman Antor	230102034	UI/UX Designer & Frontend Developer
05	Shouvik Biswas	230102024	SQA Engineer & Business Analyst

Department of Computer Science & Engineering

Table of Contents

0. Contributions:

Each member of **Team Innovex** contributed to this SRS document according to their assigned role within the project. The table below outlines individual contributions to both the project and this document.

Member	Role	SRS Contribution
Afsana	Team Leader	Overall document coordination, problem statement, conclusion, stakeholder identification
Sabiha Akter	Database Administrator	ERD, structural modeling, data flow diagrams, class-based modeling
Tasnim Ahmed Arnob	Backend Engineer	Use case modeling, sequence diagrams, activity diagrams, elicitation
Mahbubur Rahman Antor	UI/UX Designer & Frontend	UI requirements, usage scenarios, QFD, behavioral modeling
Shouvik Biswas	SQA Engineer & Business Analyst	Inception, SWOT analysis, collaborative requirements, conflict resolution

1. NextHire – Problem Statement:

Most organizations particularly small and medium-sized businesses still rely on entirely manual processes to screen job applications. When a single job posting attracts hundreds of CVs collected by the company, HR teams are forced to read each one individually, a process that is slow, inconsistent, and costly in both time and resources.

The core issues with the current state of recruitment screening are:

- **Volume Overload:** A single job post can attract hundreds of CVs within days. A small HR team physically cannot review each CV in a timely manner, meaning many qualified candidates are overlooked simply due to resource constraints.
- **Inconsistency in Evaluation:** When multiple reviewers assess CVs, each applies slightly different standards. This leads to uneven evaluations and biased decisions that are difficult to audit or justify.
- **Prolonged Time-to-Interview:** Manual screening stretches the hiring timeline into weeks. Strong candidates receive competing offers during this window and are lost before HR teams can reach them.
- **No Structured Ranking System:** Without a scoring or ranking mechanism, the decision of who advances is subjective and hard to defend to stakeholders.
- **Poor Scalability:** Spreadsheets and email inboxes were never designed for recruitment workflows. They fail as CV volumes grow and provide no analytical insight into the hiring pipeline.

These compounded inefficiencies directly reduce the quality of hiring decisions and damage organizations' ability to attract top talent. NextHire is designed to address each of these failure points through an intelligent, automated, and scalable recruitment platform used exclusively by hiring organizations. Companies collect CVs from candidates through their own existing channels (email, walk-in, job boards, etc.) and upload those CVs into NextHire, which then screens, scores, and ranks them automatically.

2. Inception:

2.1 Introduction

The inception phase of NextHire began with a focused investigation into the recruitment domain to understand how hiring actually works in practice — and where it systematically fails. The team conducted informal interviews with HR personnel, reviewed published research on recruitment automation, and analyzed competitor platforms to understand the landscape.

From this research, it became clear that the problem was not just a technology gap but a workflow gap. Organizations did not lack tools; they lacked tools tailored to the specific, structured demands of modern high-volume recruitment. NextHire was conceived as a platform that bridges this gap by embedding AI into each stage of the screening pipeline. NextHire is strictly a company-facing, internal screening tool: candidates never log in, register, or interact with the platform directly. Companies are responsible for sourcing CVs and uploading them into the system on the candidates' behalf.

2.2 Identifying Stakeholders

The following stakeholders were identified as directly involved in or affected by the NextHire system:

2.2.1 Organization / Company Admin

The hiring organization that registers on NextHire is the primary client stakeholder. They need a reliable, secure, and configurable platform to manage their job postings, upload candidate CVs, review AI-filtered candidates, and make final hiring decisions. Their key concerns are data privacy, ease of use, and cost-efficiency.

2.2.2 HR Manager / Recruiter

HR managers interact most frequently with the platform. They create job postings, define required skills and qualifications, upload CVs collected from candidates through their own channels, review AI-ranked candidates, and use the shortlisting tools. They need intuitive dashboards, fast filtering, and trustworthy AI recommendations to do their jobs effectively.

2.2.3 System Administrator

The system administrator is responsible for maintaining the NextHire platform infrastructure — managing server health, user accounts, security patches, and system configuration. They require back-end access tools and logging capabilities.

2.2.4 Department Heads / Hiring Managers

Department heads review shortlisted candidates forwarded by HR. They use the platform primarily to review AI-generated summaries and scores, then provide final acceptance or rejection feedback.

2.2.5 System Developers

Team Innovex members who design, build, test, and maintain the NextHire platform. They are responsible for ensuring the system meets all functional and non-functional requirements defined in this document.

2.2.6 Institution / University

As this project originates from an academic context, the university department acts as a supervising stakeholder with interest in the technical rigor, documentation quality, and learning outcomes achieved through the project.

2.3 Recognizing Multiple Viewpoints

Different stakeholders hold different — and sometimes conflicting — viewpoints about what the system should prioritize. Recognizing these early helps design a balanced platform:

Stakeholder	Primary Concern	Key Viewpoint
HR Manager	Speed and efficiency of screening	Wants AI to do the heavy lifting; trusts scores to shortlist
Company Admin	Data privacy and cost	Wants secure multi-tenant isolation and affordable pricing
Dept. Head	Quality of shortlist	Wants only the best-matched candidates presented, with clear rationale
System Admin	Stability and security	Wants robust logs, monitoring, and admin controls
Developers	Technical feasibility	Want clear requirements with testable acceptance criteria

2.4 Working Towards Collaboration

2.4.1 Common Requirements

- All stakeholders agree on the need for a secure, reliable platform with role-based access.
- Fast, accurate candidate ranking is valued by both HR managers and hiring departments.
- Clean, intuitive UI is desired by all non-technical users.
- Data integrity and backup capability are universally required.

2.4.2 Conflicting Requirements

- HR managers want aggressive automated shortlisting; hiring managers prefer to review larger candidate pools.

- Developers prefer incremental AI integration; stakeholders expect full AI features from launch.

2.4.3 Final Requirements Resolution

Conflicts were resolved by: (a) providing score breakdowns in a collapsible detail panel visible to HR, (b) making the shortlist size configurable per job post, and (c) implementing AI features in a microservice architecture allowing phased rollout without disrupting the core platform.

2.5 Asking the First Questions

The inception phase surfaced the following foundational questions that guided the requirements process:

- Who are the primary users, and how technically comfortable are they?
- What volume of CVs must the system handle without degrading performance?
- How should the AI scoring model be trained, validated, and updated?
- What integrations (email, calendar, third-party HR tools) are needed at launch vs. later?
- How will the system handle edge cases — CVs with non-standard formats, missing fields, or unusual layouts?
- What data privacy regulations apply to candidate information uploaded by companies in our target markets?

2.6 Conclusion

The inception phase confirmed that NextHire addresses a genuine, widespread problem with a well-defined stakeholder community and clear business value. The team established a shared understanding of goals, identified potential conflicts early, and defined the foundational questions that would drive the elicitation phase. The project is feasible, impactful, and achievable within the planned one-semester timeline.

3. Elicitation:

3.1 Introduction

Requirements elicitation is the process of discovering, extracting, and refining what the system must do from the perspectives of all relevant stakeholders. For NextHire, elicitation activities included structured interviews with simulated HR professionals, competitive analysis of existing recruitment tools, review of academic literature on NLP-based resume screening, and collaborative team workshops.

3.2 Eliciting Requirements

The following elicitation techniques were applied:

- **Interviews:** Informal conversations with CS department staff and acquaintances in HR roles to understand day-to-day hiring pain points.
- **Observation:** Review of recruitment workflows described in public case studies and HR industry reports.
- **Competitive Analysis:** Evaluation of platforms such as Workable, Lever, and Greenhouse to identify feature gaps NextHire can address.
- **Brainstorming Sessions:** Weekly team workshops to generate and prioritize feature ideas.
- **Prototype Feedback:** Paper and Figma mockups were reviewed internally to validate UI assumptions.

3.3 Collaborative Requirements Gathering

Requirements were gathered collaboratively across three categories: what users need the system to do (functional), how well it must perform (non-functional), and what business constraints it must respect. Each requirement was tagged with a priority (Must Have, Should Have, Nice to Have) and assigned an owner for traceability.

3.4 Quality Function Deployment (QFD)

3.4.1 Normal Requirements

Requirements that users explicitly state and expect as standard features:

- Bulk CV upload and structured storage by HR/Company Admin
- Job posting creation with skill and experience parameters
- Candidate ranking dashboard
- Role-based login (Admin, HR, Department Head)
- CV processing status tracking

3.4.2 Expected Requirements

Requirements users assume without stating their absence causes dissatisfaction:

- System must not lose uploaded CV data under any circumstances

- Scores must be deterministic for the same CV and job post combination
- Platform must work on major browsers without plugin installation
- All forms must validate input and provide clear error messages

3.4.3 Exciting Requirements

Requirements users do not expect but will delight them if present:

- AI-generated plain-English summary of each candidate's CV highlights
- Smart shortlisting that learns from recruiter feedback over time
- Internal team messaging scoped to individual job posts
- Analytics dashboard showing hiring funnel metrics and bottleneck identification

3.5 Usage Scenarios

3.5.1 Viewing Available Candidates

Scenario: An HR manager at a mid-size company has just finished collecting CVs for a Software Engineer role from various sources. She uploads the batch of CVs into NextHire, navigates to the job post, and the dashboard instantly displays ranked candidates sorted by AI match score. She filters to view only candidates scoring above 75%, reviews AI-generated summaries for the top 10, and shortlists 5 for interview all within 20 minutes, compared to the 3 days the manual process previously required.

3.5.2 Handling an Incomplete CV

Scenario: An uploaded CV is missing the education section. The AI parsing module flags the missing fields with a warning tag visible to the HR manager. The system still generates a partial match score based on available data and appends a note: 'Education data unavailable score reflects skills and experience only.' The HR manager can then decide whether to contact the candidate directly through their own channels for the missing information.

3.6 Elicitation Work Products

The elicitation phase produced the following documented artifacts:

- Stakeholder register with roles, interests, and influence levels
- Prioritized feature backlog (Trello board, 40 user stories)
- Use case inventory (10 primary use cases, 22 sub-cases)
- QFD matrix mapping stakeholder needs to system features
- Figma low-fidelity wireframes for the core HR dashboard
- Data dictionary draft (13 primary entities identified)

4. Use Case Modeling:

4.1 Use Case Summary Table

UC-ID	Use Case Name	Actor(s)	Priority
UC-01	Register Organization	Company Admin	Must Have
UC-02	Login / Logout	All Users	Must Have
UC-03	Manage User Accounts	System Admin	Must Have
UC-04	Create Job Posting	HR Manager	Must Have
UC-05	Upload Candidate CVs	HR Manager	Must Have
UC-06	AI Resume Screening	System (Auto)	Must Have
UC-07	View Ranked Candidates	HR Manager	Must Have
UC-08	Shortlist Candidates	HR Manager	Must Have
UC-09	Review Candidate Summary	HR / Dept. Head	Must Have
UC-10	Send Internal Message	HR Manager	Should Have
UC-11	View Analytics Dashboard	HR / Admin	Should Have
UC-12	Submit Platform Feedback	Any User	Nice to Have

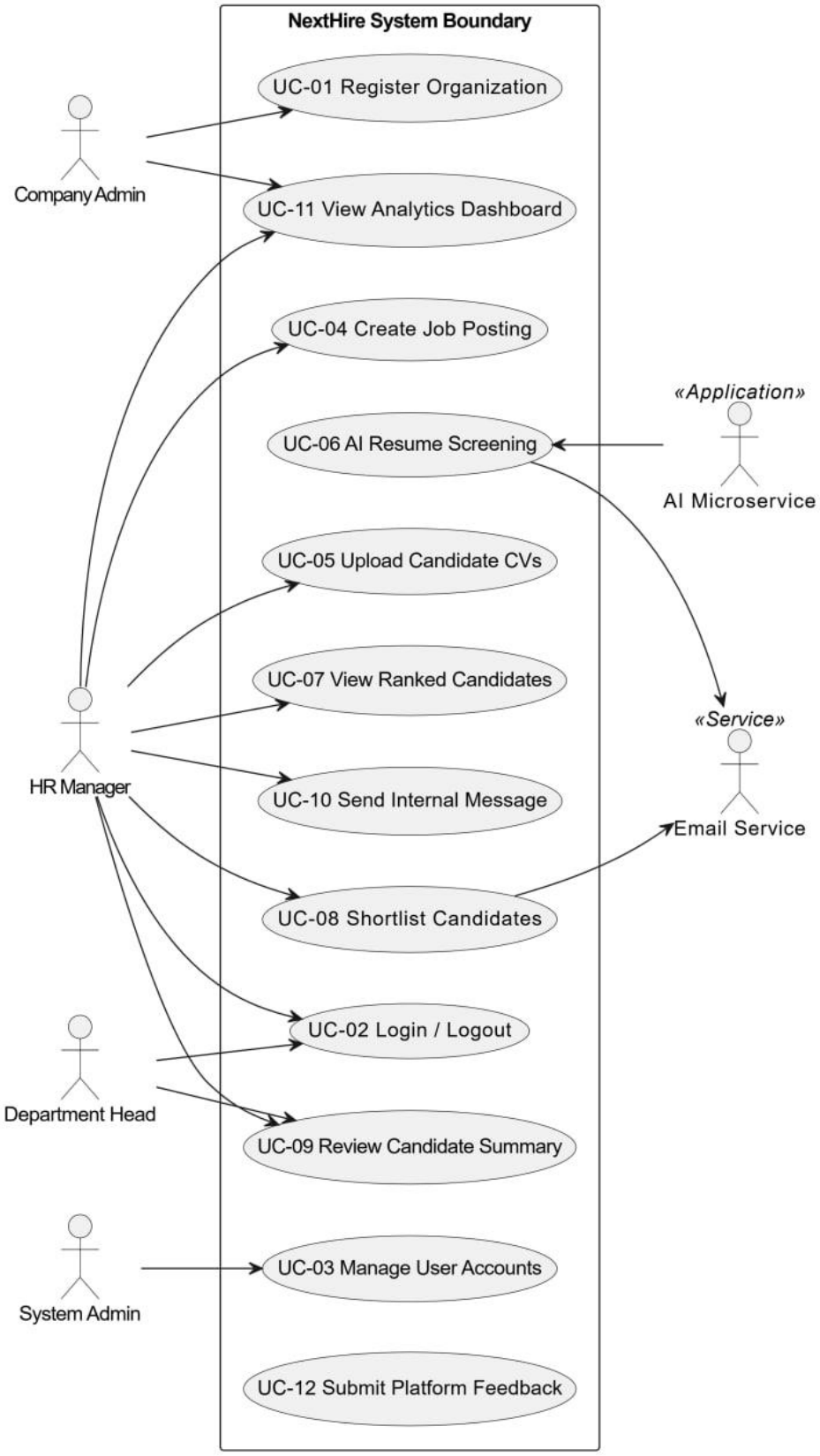
4.2 Use Case Scenario

Primary scenario for UC-06: AI Resume Screening

- **Precondition:** A job post exists with defined required skills, experience level, and qualifications. At least one CV has been uploaded by the company.
- **Trigger:** HR manager uploads one or more CVs OR manually initiates batch screening.
- **Main Flow:** System extracts text from uploaded CV. NLP pipeline parses skills, education, and experience. Match engine compares parsed data against job criteria. A percentage match score is calculated and stored. An AI-generated summary is created and attached to the candidate profile. Candidate ranking is updated on the dashboard.
- **Alternate Flow:** If a CV is in an unsupported format, the system returns an error and notifies the uploading HR user to re-upload in PDF or DOCX.
- **Post condition:** Candidate appears in the ranked list with score and AI summary visible to HR manager.

4.3 Use Case Diagram

The NextHire use case hierarchy spans two levels. Level 0 defines the system boundary; Level 1 expands each functional domain into specific use cases. Due to document format constraints, the diagram is described textually below and rendered visually in the companion Figma design file.


NextHire System Boundary
UC-01 Register Organization
UC-11 View Analytics Dashboard
UC-04 Create Job Posting
UC-06 AI Resume Screening
UC-05 Upload Candidate CVs
UC-07 View Ranked Candidates
UC-10 Send Internal Message
UC-08 Shortlist Candidates
UC-02 Login / Logout
UC-09 Review Candidate Summary
UC-03 Manage User Accounts
UC-12 Submit Platform Feedback
Company Admin
HR Manager
Department Head
System Admin
«Application»
AI Microservice
«Service»
Email Service

4.4 Use Case Descriptions

4.4.1 Level 0 – NextHire System

The Level 0 diagram defines the entire NextHire platform as a single system unit. External actors — Company Admin, HR Manager, System Admin, and the AI Microservice — interact with the system boundary. All interactions pass through the authentication layer before reaching any functional module.

4.4.2 Level 1 – Core System Functionalities

Level 1 decomposes the system into eight functional domains: Authentication, User Management, Job Management, CV Management, AI Screening, Candidate Ranking, Messaging, and Analytics.

4.4.3 Level 1.1 – Authentication

- **Login:** All users authenticate via email and password with JWT token issuance.
- **Logout:** Session token is invalidated server-side on logout.
- **Password Reset:** OTP sent via email; new password must meet complexity policy.
- **Role Assignment:** Roles (Admin, HR, Dept. Head) assigned on registration.

4.4.4 Level 1.2 – User Management

- **Register Organization:** Company Admin creates a new tenant account with organization details.
- **Invite HR User:** Admin sends email invitation to add HR manager accounts.
- **Deactivate Account:** Admin can suspend user access without deleting data.
- **View Audit Log:** Admin can review all system actions by user and timestamp.

4.4.5 Level 1.3 – Job & CV Management

- **Create Job Post:** HR defines title, description, required skills, experience level, and interview quota.
- **Publish / Unpublished Job:** Controls the active status of a job post within the platform.
- **Upload CV(s):** HR uploads one or more candidate CVs in PDF/DOCX format against a job post, either individually or in bulk.
- **Track Processing Status:** HR can view the current screening status of each uploaded CV (Queued, Processing, Screened, Failed).

4.4.6 Level 1.4 – AI Screening & Ranking

- **Parse Resume:** Python NLP microservice extracts structured data from uploaded CVs.
- **Calculate Match Score:** Scoring engine computes weighted percentage based on skills, experience, and education alignment.
- **Generate AI Summary:** LLM-based summarizer produces a 3–5 sentence candidate highlight for each CV.
- **Rank Candidates:** All CVs for a job post are sorted by score, with ties broken by upload timestamp.

4.4.7 Level 1.5 – Shortlisting & Decisions

- **Smart Shortlist:** System automatically marks the top N candidates (N defined per job post) as shortlisted.
- **Manual Override:** HR manager can include or exclude specific candidates regardless of AI score.
- **Accept / Reject:** HR or Dept. Head marks final decision per candidate.
- **Record Decision:** System logs the final decision against the candidate record for internal reporting.

4.4.8 Level 1.6 – Messaging, Notifications & Feedback

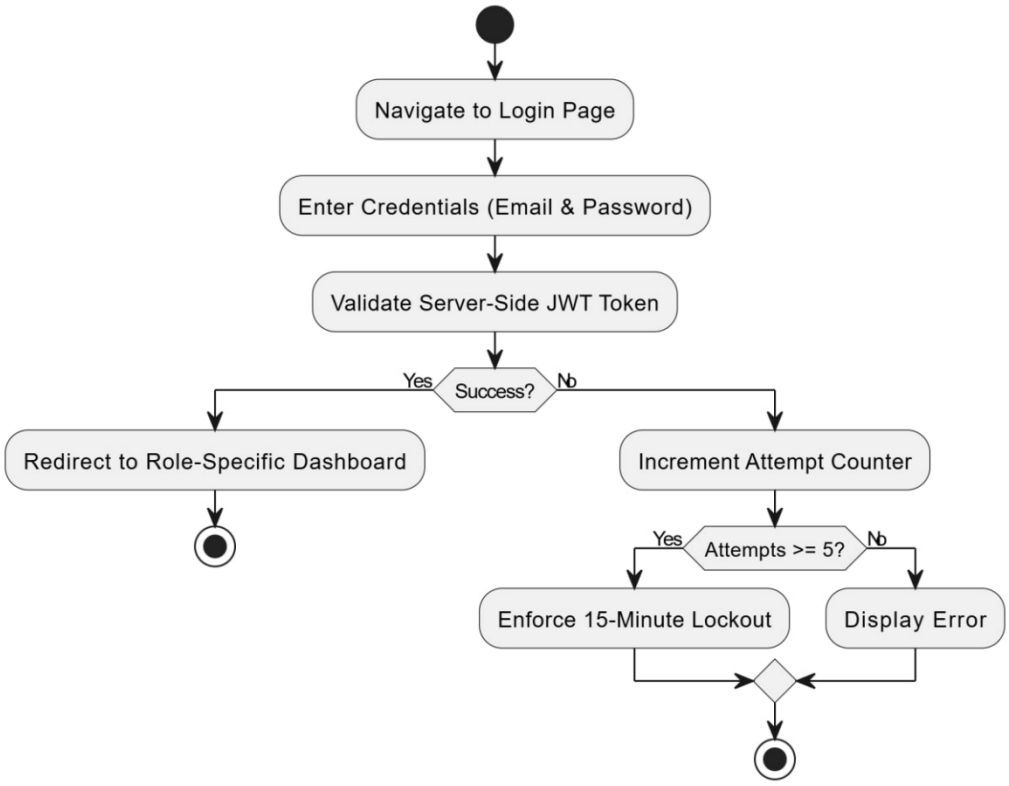
- **Internal Chat:** HR team members can exchange messages in a thread scoped to each job post.
- **Recruitment Dashboard:** Displays KPIs including total CVs processed, shortlist rate, average score, and time-to-shortlist.
- **Export Report:** HR can export candidate tables as CSV for external review.
- **Platform Feedback:** Users can submit star ratings and text suggestions; Admin reviews all submissions.

5. Activity Diagrams:

Activity diagrams model the workflow of each functional area as a sequence of actions, decisions, and parallel flows. The following sections describe the logic captured in each diagram.

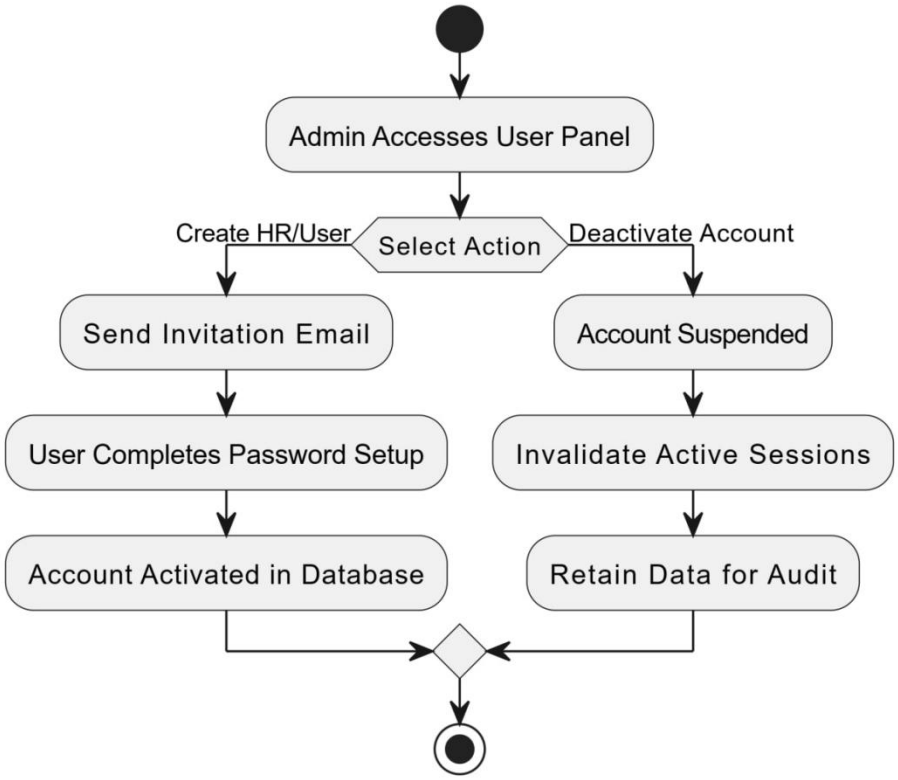
5.1 Authentication

Flow: User navigates to login page → Enters credentials → System validates JWT token → On success, redirect to role-specific dashboard → On failure, increment attempt counter → After 5 failures, enforce 15-minute lockout.



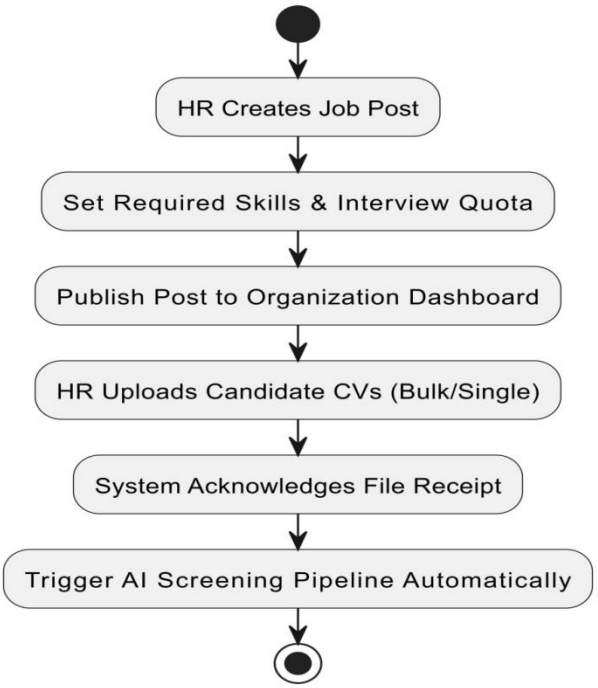
5.2 User Management

Flow: Admin accesses user panel → Selects action (Create, Edit, Deactivate) → For Create: sends invitation email → User completes registration → Account activated → For Deactivate: account suspended, active sessions invalidated, data retained.



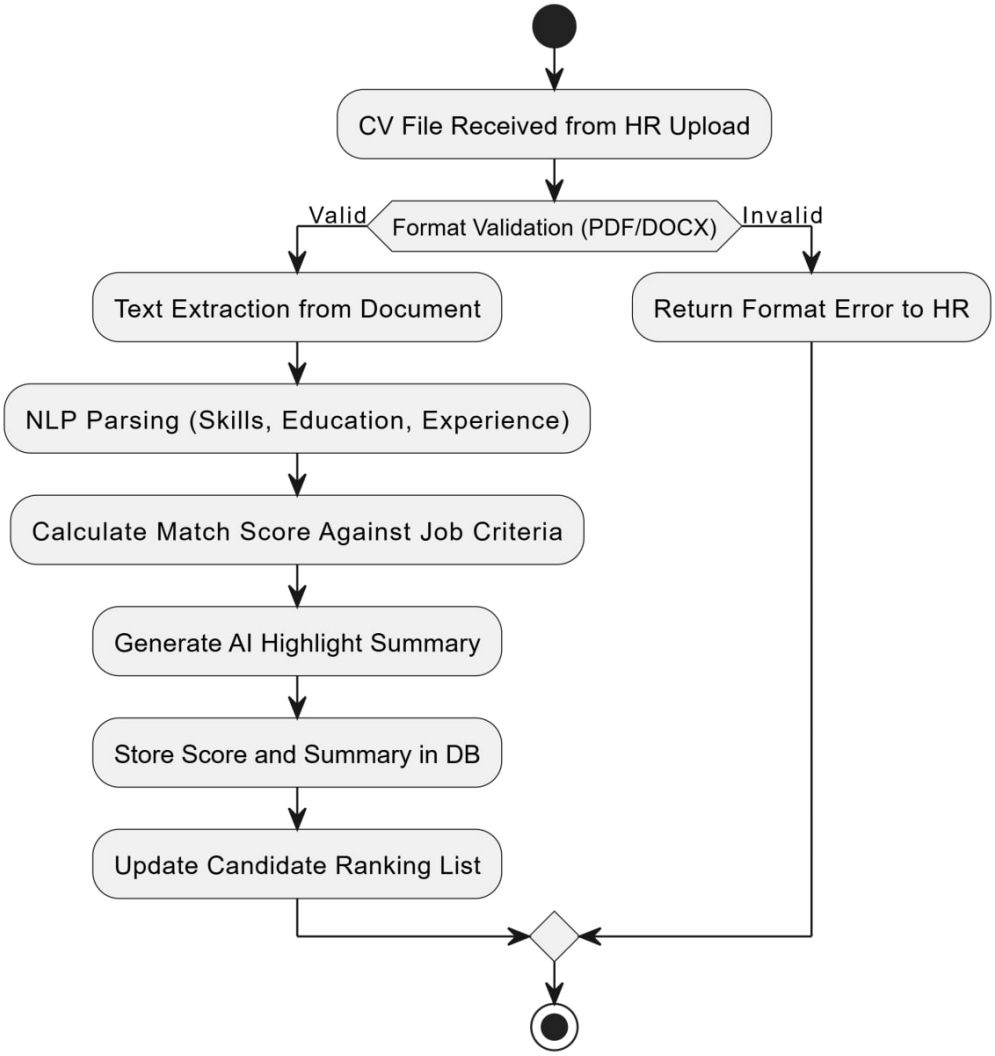
5.3 Job & CV Management

Flow: HR creates job post → Sets skill requirements and interview quota → Publishes post → HR uploads one or more candidate CVs against the job post → System acknowledges receipt → AI screening pipeline triggered automatically for each uploaded CV.



5.4 AI Screening Pipeline

Flow: CV received → Format validation (PDF/DOCX) → Text extraction → NLP parsing (skills, education, experience) → Match score calculation against job criteria → AI summary generation → Score and summary stored → Candidate ranking updated on dashboard.

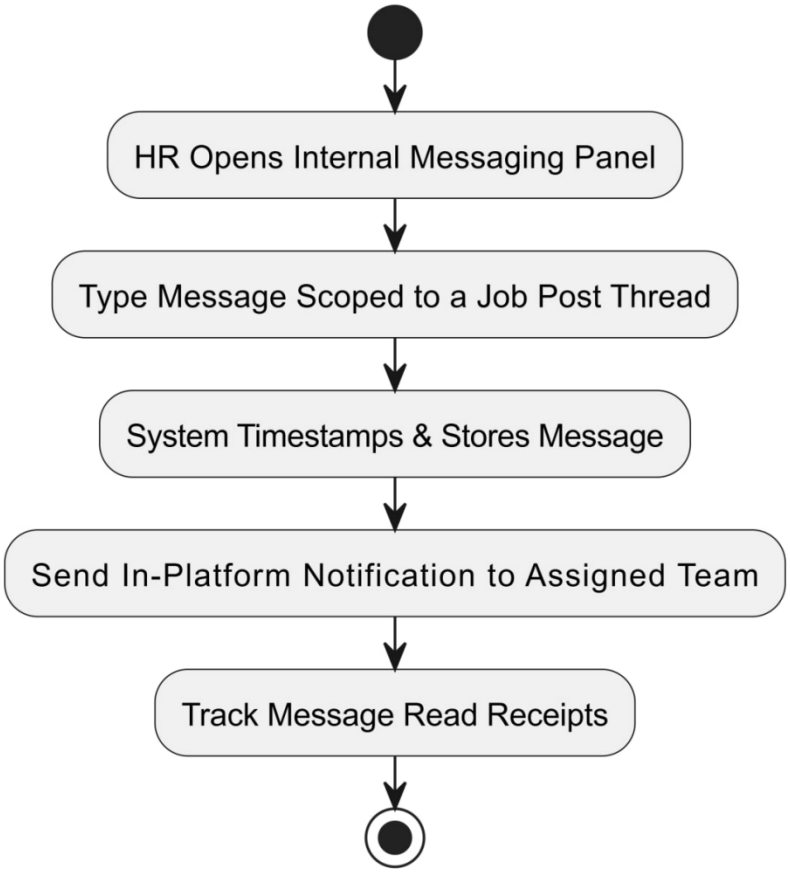


5.5 Candidate Ranking & Shortlisting

Flow: HR opens job post dashboard → Views ranked candidate list → Reviews AI summaries → Applies score threshold filter → System highlights auto-shortlist → HR confirms or adjusts shortlist → Candidate status updated → Decision recorded for internal records.

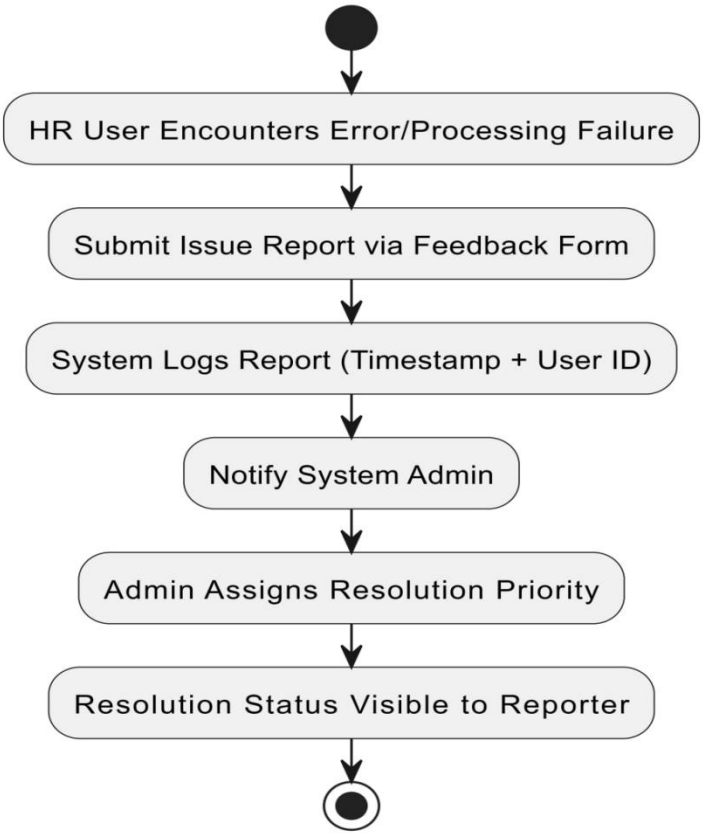
5.6 Internal Messaging

Flow: HR opens internal messaging panel for a job post → Types message → System timestamps and stores message → All HR members with access to that job post receive an in-platform notification → Read receipts tracked.



5.7 Incident Reporting (System Issues)

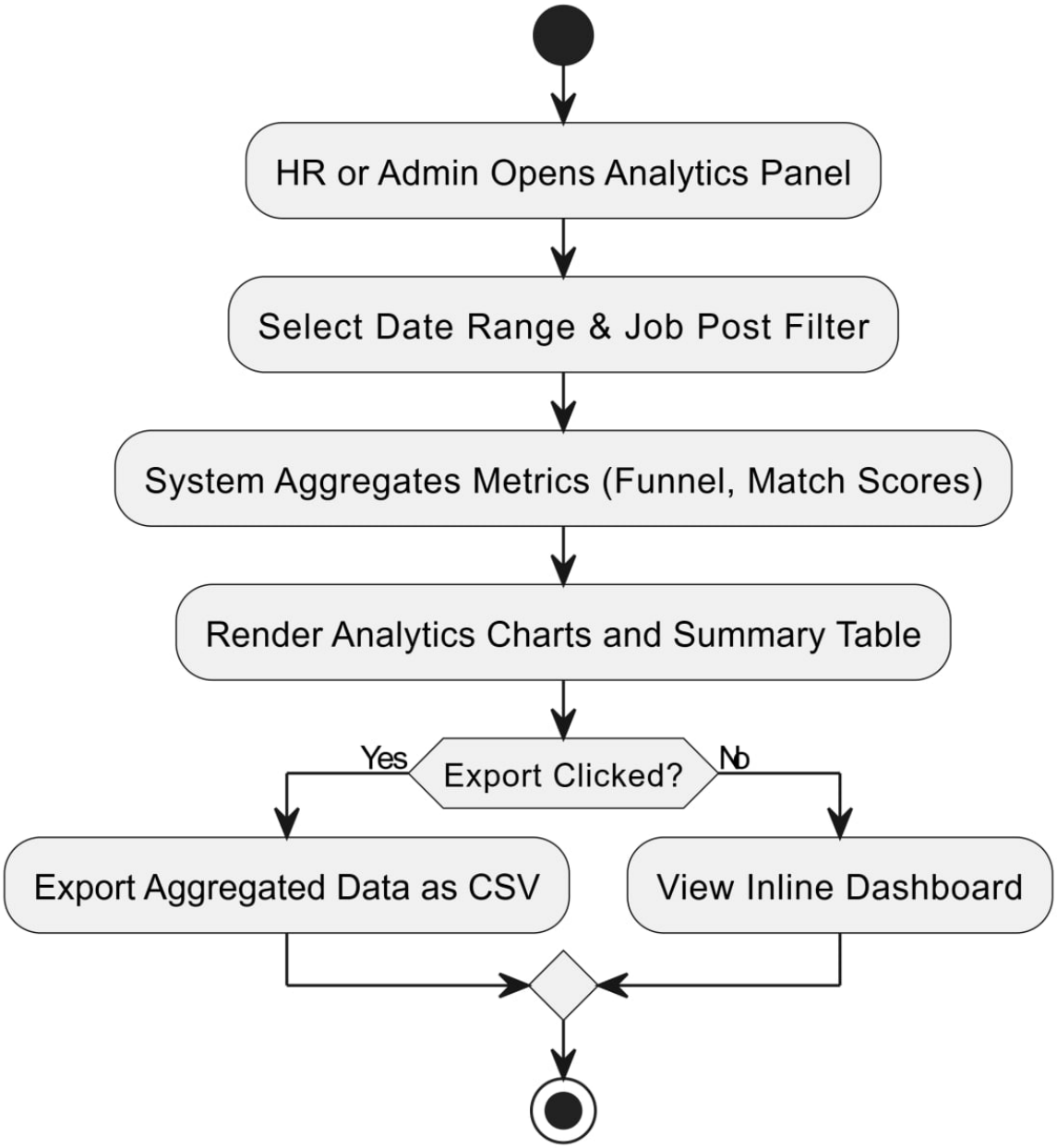
Flow: HR user encounters a system error or CV processing failure → Submits issue report via feedback form → System logs report with timestamp and user ID → Admin notified → Admin assigns resolution priority → Resolution status visible to reporter



.

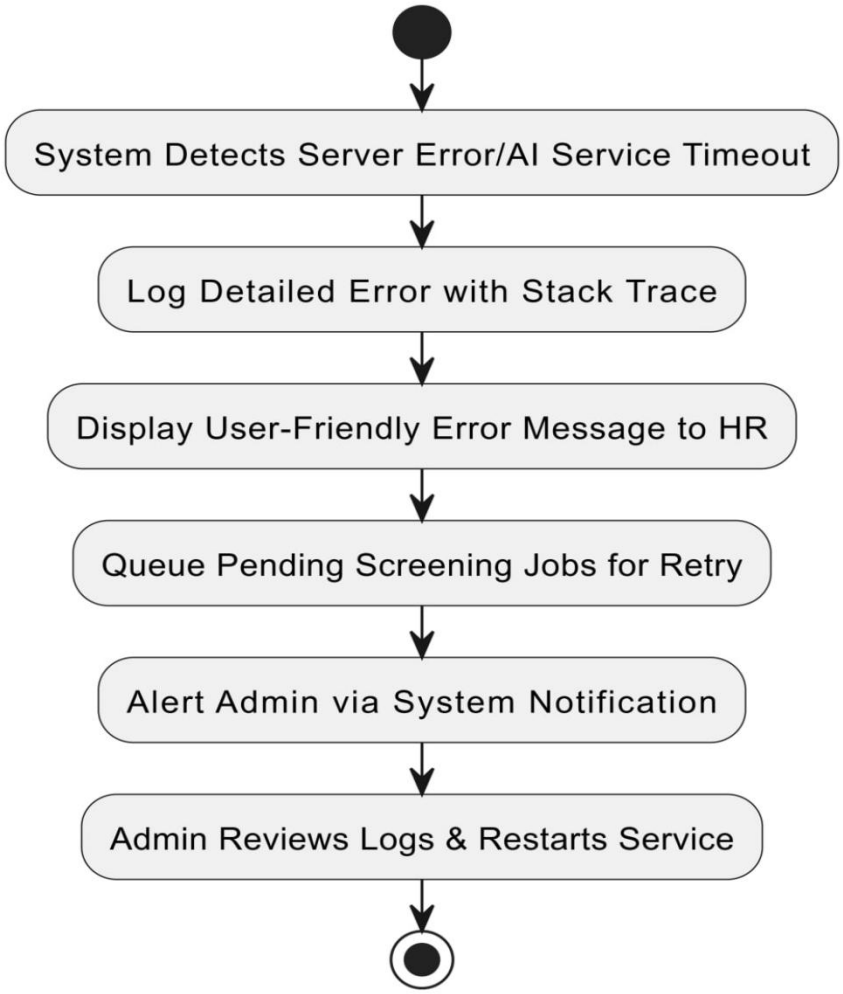
5.8 Analytics

Flow: HR or Admin opens analytics panel → Selects date range and job post filter → System aggregates: total CVs processed, shortlist rate, average match score, time-to-shortlist, rejection rate → Renders charts and summary table → User optionally exports as CSV.



5.9 Emergency / Critical Failure Management

Flow: System detects server error or AI microservice timeout → Logs error with stack trace → Displays user-friendly error message → Queues pending screening jobs for retry → Admin alerted via system notification → Admin reviews logs and restarts service if needed.

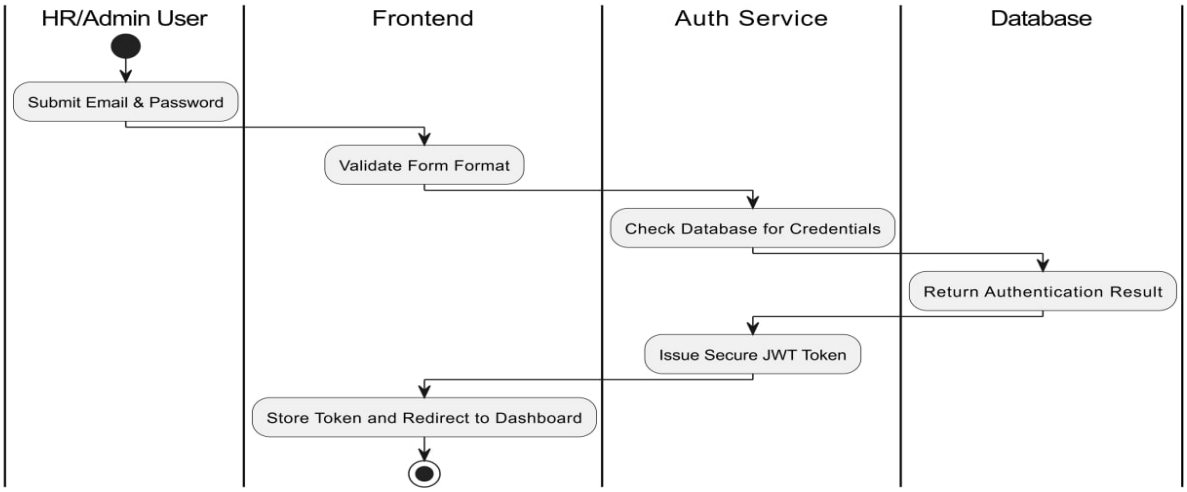


6. Swimlane Diagrams:

Swimlane diagrams extend activity diagrams by assigning each action to a specific actor lane, clarifying responsibility boundaries. The following sections describe the actor lanes and flow for each functional area.

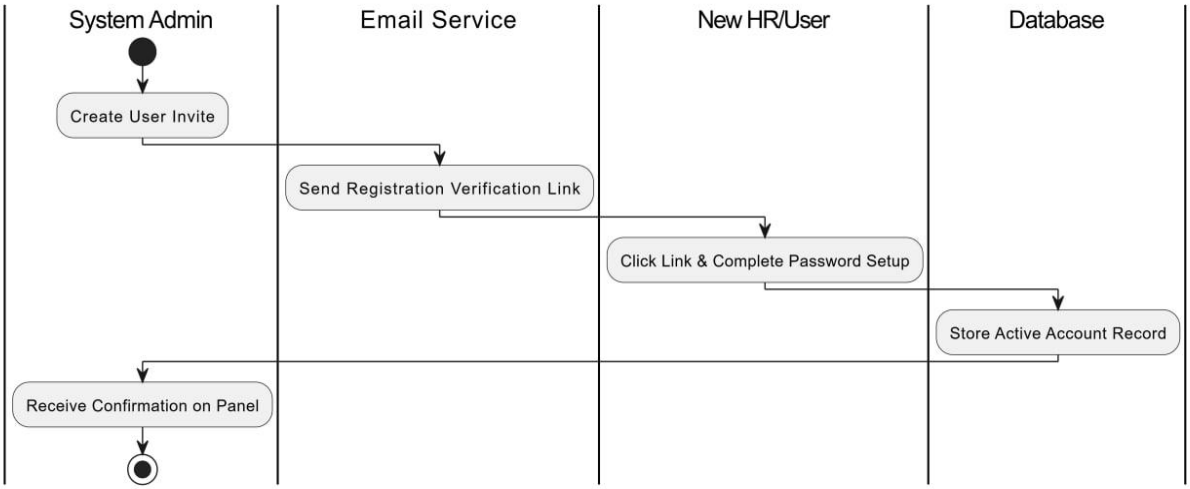
6.1 Authentication

Lanes: User | Frontend | Auth Service | Database. User submits credentials (User lane) → Frontend validates form format → Auth Service checks database for matching hash → Database returns result → Auth Service issues JWT → Frontend stores token and redirects.



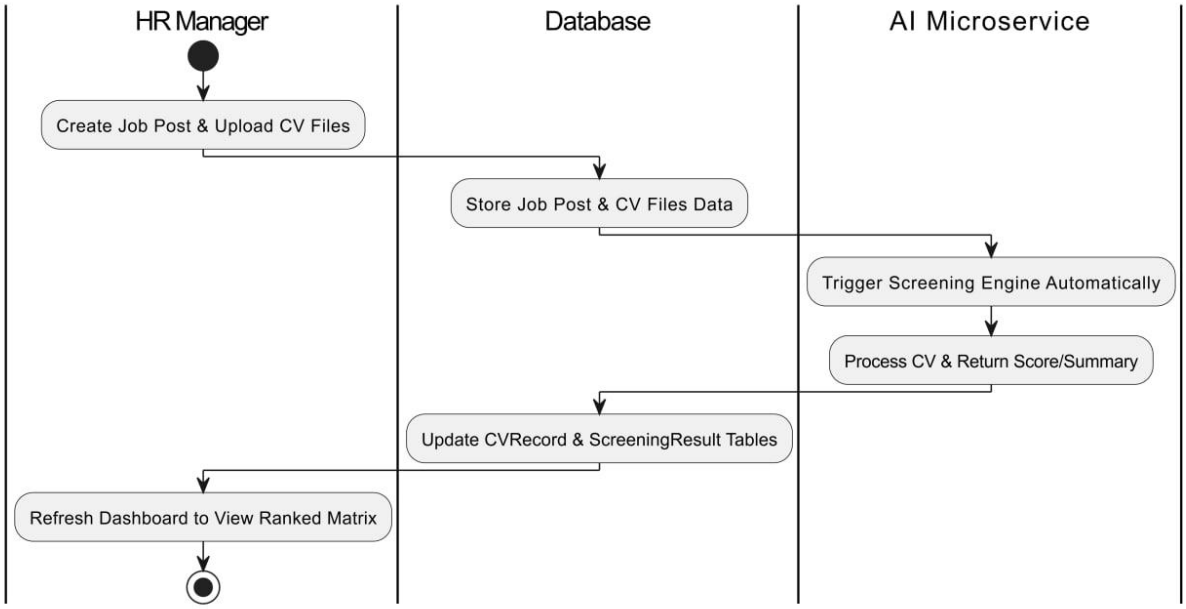
6.2 User Management

Lanes: System Admin | Email Service | New User | Database. Admin creates invite (Admin lane) → Email Service sends link → New User clicks link and completes registration → Database stores new account record → Admin receives confirmation.



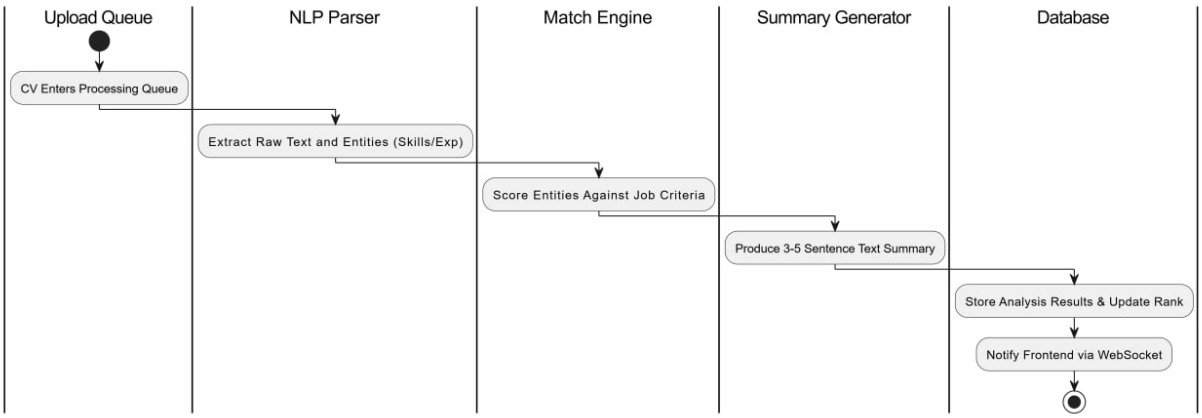
6.3 Job & CV Management

Lanes: HR Manager | AI Microservice | Database | Notification Service. HR creates post (HR lane) → Database stores job → HR uploads candidate CV(s) → Database stores CV → AI Microservice triggered → Microservice returns score and summary → Database updated → HR dashboard refreshed.



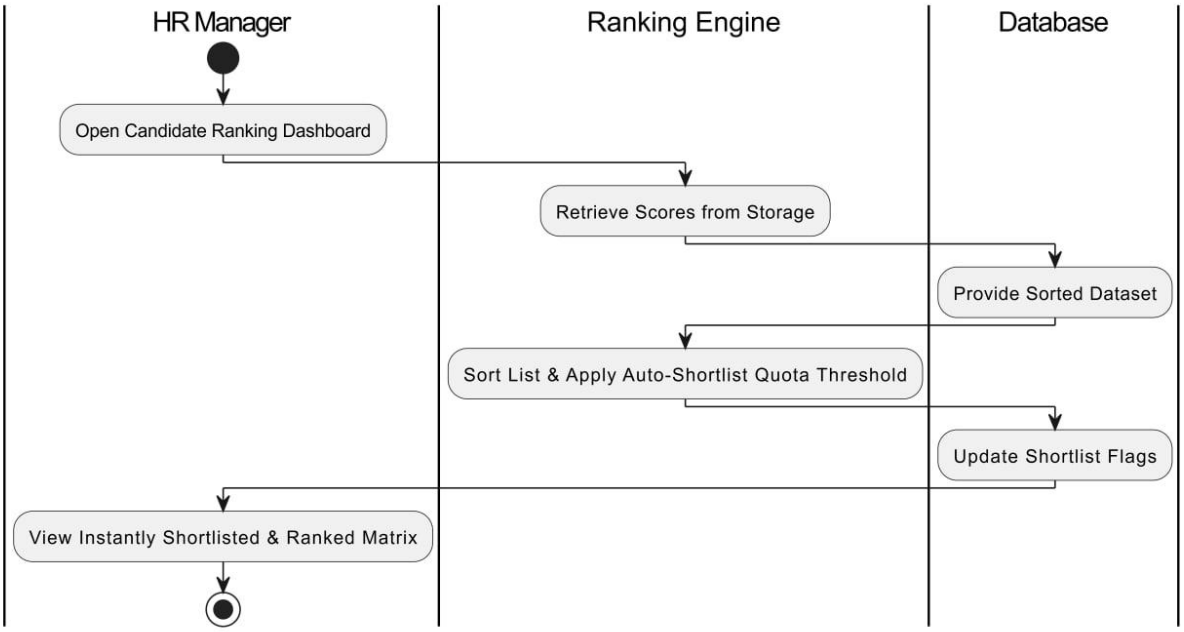
6.4 AI Screening

Lanes: Upload Queue | NLP Parser | Match Engine | Summary Generator | Database. CV enters queue → NLP Parser extracts entities → Match Engine scores against job criteria → Summary Generator produces text → All results stored in Database → Frontend notified of completion.



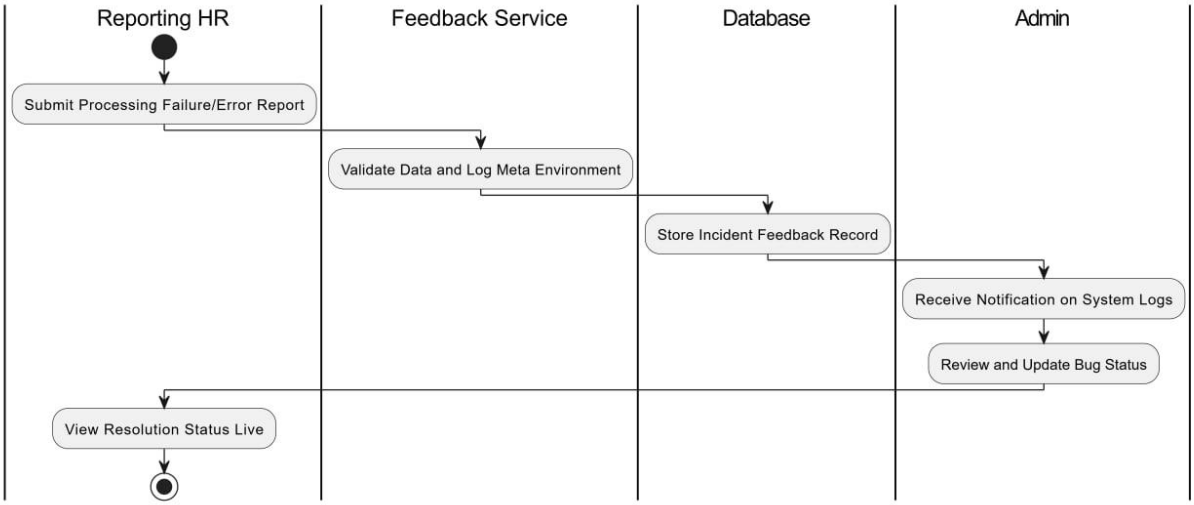
6.5 Candidate Ranking

Lanes: HR Manager | Ranking Engine | Database. HR triggers ranking review → Ranking Engine retrieves all scores from Database → Sorts by score → Applies auto-shortlist threshold → Database updated with shortlist flags → HR dashboard displays shortlisted candidates.



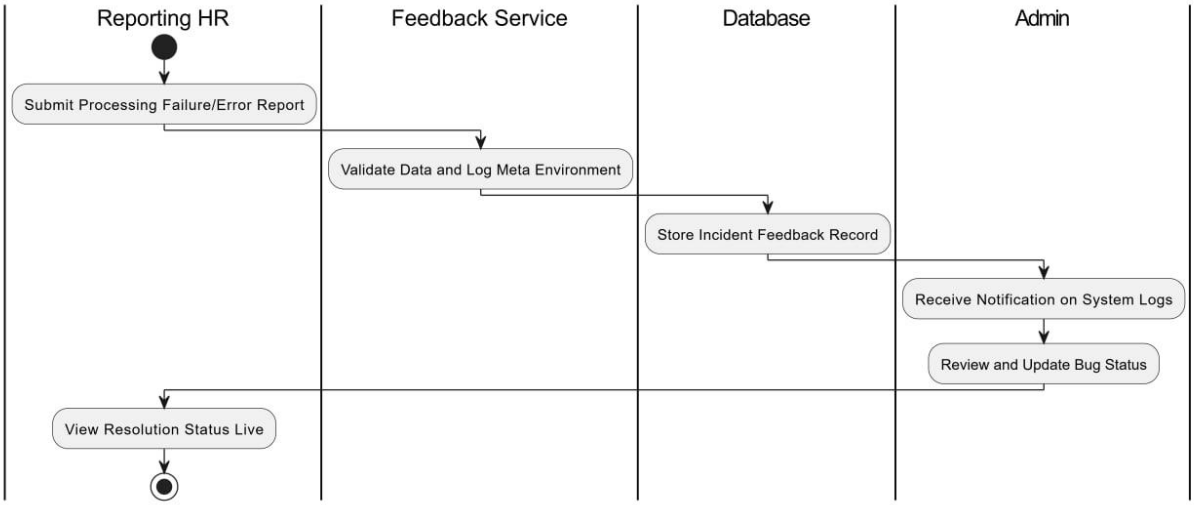
6.6 Messaging

Lanes: HR User A | HR User B | Messaging Service | Database. User A sends message → Messaging Service persists to Database → User B receives in-app notification → User B reads and replies → Conversation thread updated.



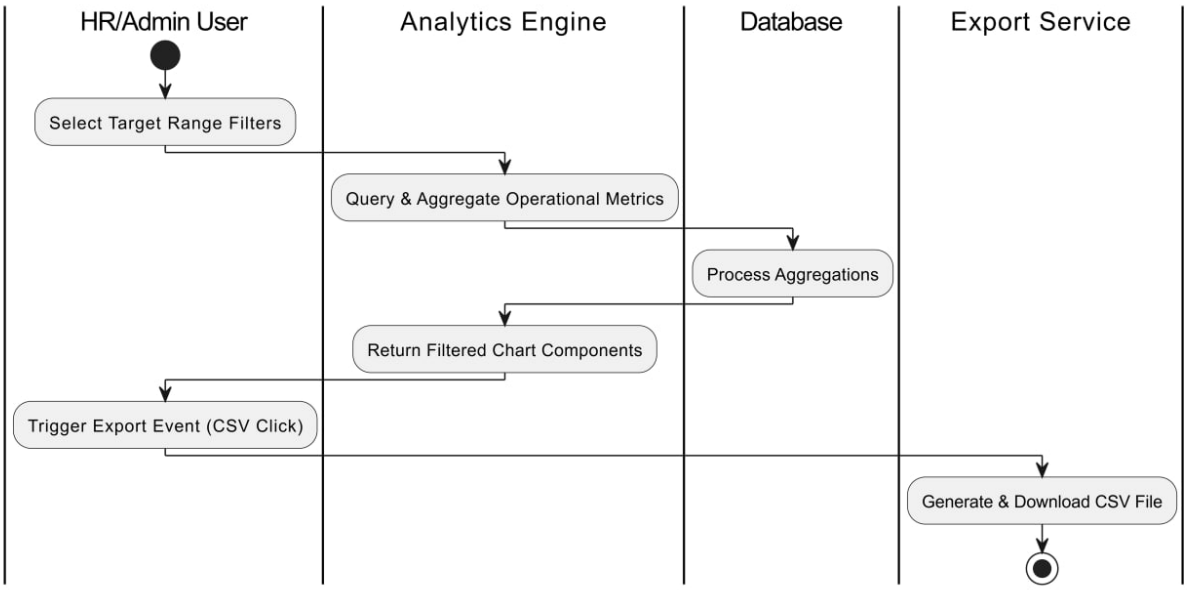
6.7 Issue Reporting

Lanes: Reporting User | Feedback Service | Admin | Database. User submits report → Feedback Service validates and stores in Database → Admin receives notification → Admin reviews and updates status → System notifies reporter of resolution.



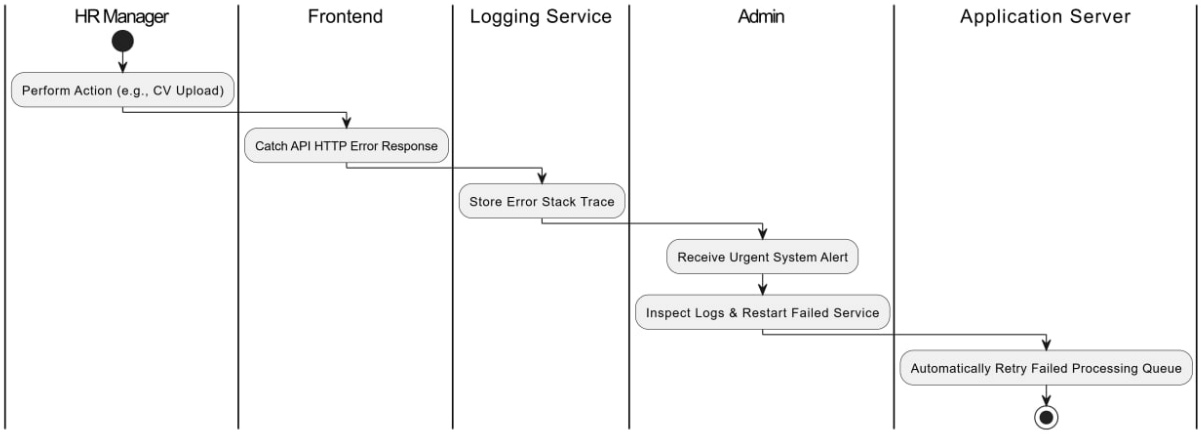
6.8 Analytics

Lanes: HR/Admin | Analytics Engine | Database | Export Service. User selects filters → Analytics Engine queries Database → Aggregates metrics → Returns data to frontend → User views dashboard → Optionally triggers Export Service for CSV download.



6.9 Critical Error Handling

Lanes: User | Frontend | Application Server | AI Microservice | Admin | Logging Service. User action triggers failure → Frontend catches HTTP error → Logging Service stores error detail → Admin receives alert → Admin inspects logs and takes corrective action → Pending queue retried automatically.



7. Structural Modeling:

7.1 Relationship Between Objects

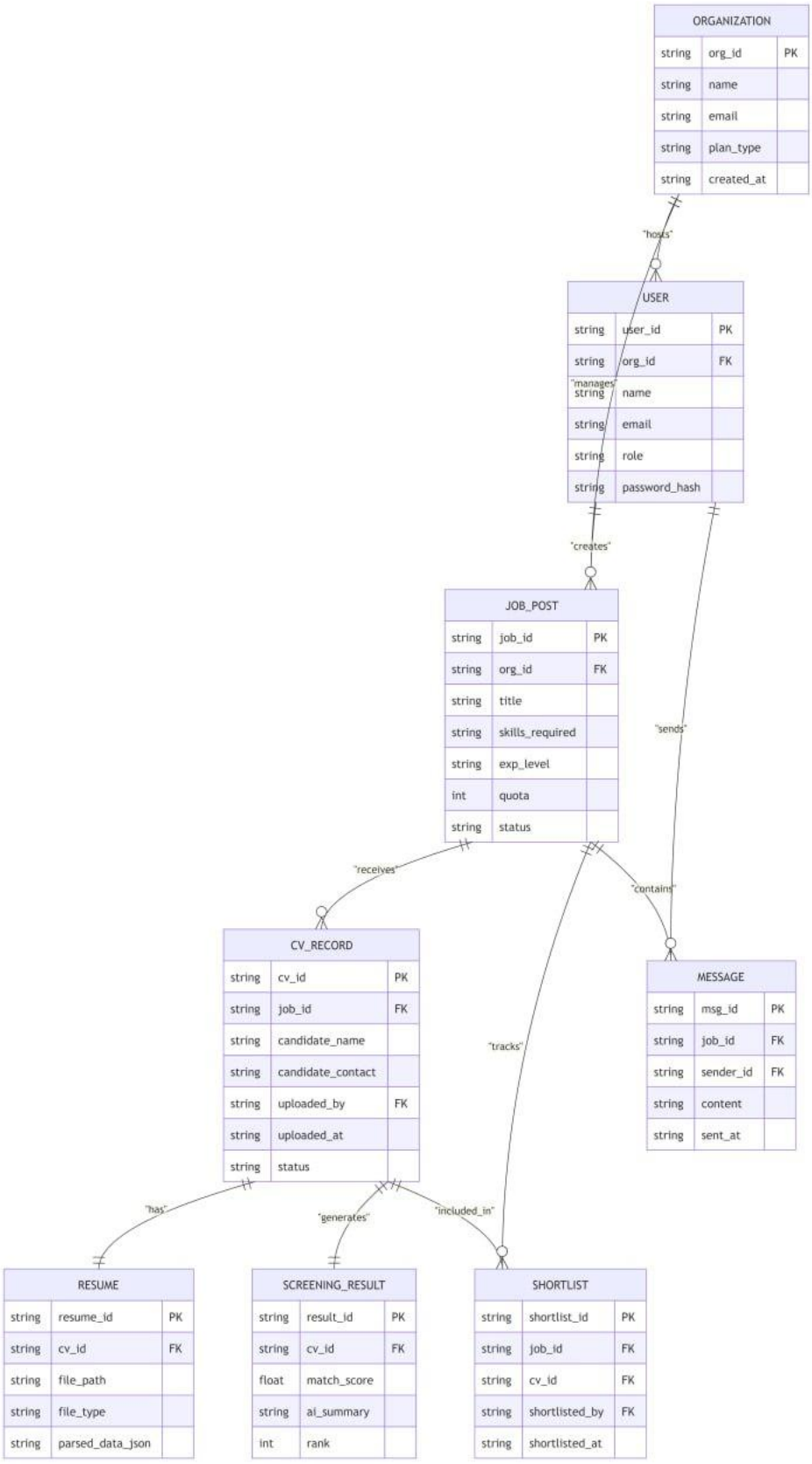
The NextHire data model is built around five core relationships:

- An Organization has many Job Posts and many Users (HR Managers, Admins).
- A Job Post has many CV Records uploaded by the company.
- Each CV Record corresponds to exactly one Resume document.
- The AI Engine processes each CV Record and produces exactly one ScreeningResult.
- An HR Manager can Shortlist zero or more CV Records per Job Post, and a Job Post can have many Messages in its internal thread.

7.2 Entity Relationship Diagram (ERD)

The following entities and their primary attributes are defined for the NextHire database:

Entity	Primary Key	Key Attributes	Relationships
Organization	org_id	name, email, plan_type, created_at	Has many Users, Job Posts
User	user_id	org_id, name, email, role, password_hash	Belongs to Organization; creates Job Posts
Job Post	job_id	org_id, title, skills_required, exp_level, quota, status	Belongs to Org; has many CV Records
CV Record	cv_id	job_id, candidate_name, candidate_contact, uploaded_by, uploaded_at, status	Belongs to Job Post; uploaded by HR User
Resume	resume_id	cv_id, file_path, file_type, parsed_data_json	Belongs to CV Record
Screening Result	result_id	cv_id, match_score, ai_summary, rank	Linked to CV Record
Shortlist	shortlist_id	job_id, cv_id, shortlisted_by, shortlisted_at	Links CV Record to Job decision
Message	msg_id	job_id, sender_id, content, sent_at	Belongs to Job Post thread
Feedback	feedback_id	user_id, rating, comment, submitted_at	Belongs to User


ORGANIZATION
string org_id PK
string name
string email
string plan_type
string created_at
"hosts"
USER
string user_id PK
string org_id FK
"manages"
string name
string email
string role
string password_hash
"creates"
JOB_POST
string job_id PK
string org_id FK
string title
string skills_required
string exp_level
int quota
string status
"sends"
"receives"
"contains"
CV_RECORD
string cv_id PK
string job_id FK
string candidate_name
string candidate_contact
string uploaded_by FK
string uploaded_at
string status
MESSAGE
string msg_id PK
string job_id FK
string sender_id FK
string content
string sent_at
"tracks"
"has"
"generates"
"included_in"
RESUME
string resume_id PK
string cv_id FK
string file_path
string file_type
string parsed_data_json
SCREENING_RESULT
string result_id PK
string cv_id FK
float match_score
string ai_summary
int rank
SHORTLIST
string shortlist_id PK
string job_id FK
string cv_id FK
string shortlisted_by FK
string shortlisted_at

7.3 Class-Based Modeling

7.3.1 Noun Identification

From the requirements document and use cases, the following nouns were identified as candidate classes: Organization, User, Admin, HRManager, JobPost, CVRecord, Resume, ScreeningResult, MatchScore, AISummary, Shortlist, Message, Notification, Feedback, DashboardMetrics, AuditLog.

7.3.2 Potential Classes

All sixteen nouns above were evaluated for class candidacy based on: (a) does it have attributes? (b) does it have behaviors? (c) is it referenced by at least two other elements?

7.3.3 Selection Criteria

Entities were retained as classes if they met at least two of the three criteria. AISummary was merged into ScreeningResult. DashboardMetrics was treated as a computed view rather than a persistent class.

7.3.4 Final List of Classes

Ten classes were selected: Organization, User (abstract), Admin, HRManager, JobPost, CVRecord, Resume, ScreeningResult, Shortlist, Message, Feedback, AuditLog.

8. Class Design:

8.1 Class Cards

8.1.1 User (Abstract)

Attribute	Type	Description
user_id	UUID	Unique user identifier
name	String	Full name
email	String	Email address (login credential)
password_hash	String	Bcrypt-hashed password
role	Enum	ADMIN, HR_MANAGER, DEPT_HEAD
created_at	DateTime	Account creation timestamp

8.1.2 HRManager

Attribute / Method	Type	Description
org_id	UUID	Foreign key to Organization
createJobPost(details)	Method	Creates a new JobPost
uploadCV(jobId, file)	Method	Uploads a candidate CV against a job post
viewRankedCandidates(jobId)	Method	Returns sorted CV record list
shortlistCandidate(cvId)	Method	Marks CV record as shortlisted
sendMessage(jobId, content)	Method	Posts to job-scoped message thread

8.1.3 JobPost

Attribute	Type	Description
job_id	UUID	Primary key
title	String	Job title
skills_required	List<String>	Required skills for the role
experience_level	Enum	ENTRY, MID, SENIOR
quota	Integer	Maximum interview slots
status	Enum	DRAFT, OPEN, CLOSED
cv_records	List<CVRecord>	Linked CV records

8.1.4 CVRecord

Attribute	Type	Description
cv_id	UUID	Primary key
job_id	UUID	Foreign key to JobPost
candidate_name	String	Name extracted/entered for the candidate
candidate_contact	String	Contact details extracted from CV, if available
uploaded_by	UUID	Foreign key to HR User who uploaded the CV
resume	Resume	Linked resume document
screening_result	ScreeningResult	AI analysis output
status	Enum	QUEUED, PROCESSING, SCREENED, SHORTLISTED, REJECTED, FAILED
uploaded_at	DateTime	Upload timestamp

8.1.5 ScreeningResult

Attribute	Type	Description
result_id	UUID	Primary key
cv_id	UUID	Foreign key to CVRecord
match_score	Float	0.0 – 100.0 match percentage
ai_summary	String	3-5 sentence candidate highlight
rank	Integer	Relative rank among all CVs for the job
generated_at	DateTime	Timestamp of AI processing completion

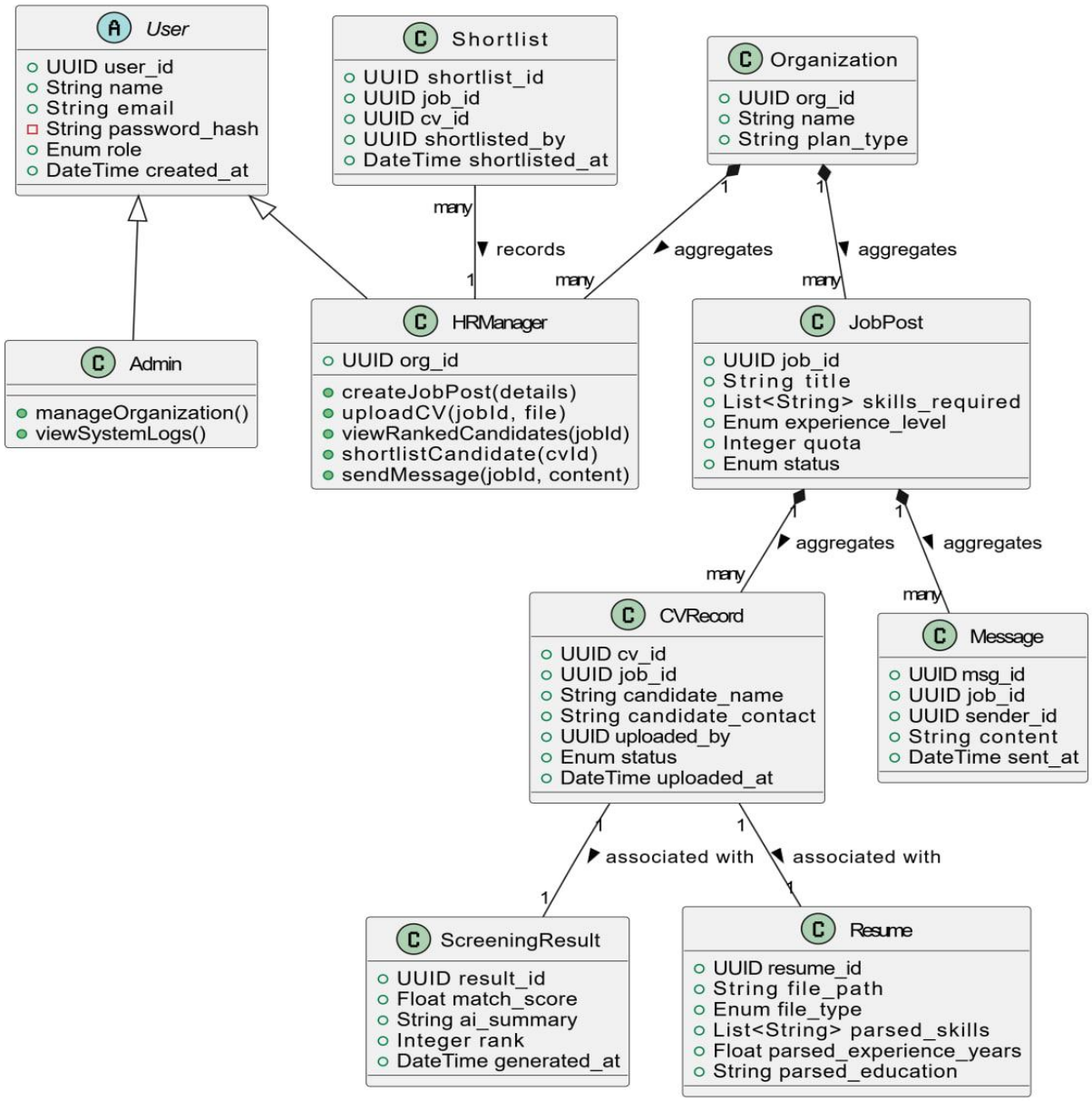
8.1.6 Resume

Attribute	Type	Description
resume_id	UUID	Primary key
cv_id	UUID	Foreign key to CVRecord
file_path	String	Secure cloud storage path
file_type	Enum	PDF, DOCX
parsed_skills	List<String>	NLP-extracted skills
parsed_experience_years	Float	Estimated years of experience
parsed_education	String	Highest qualification detected

8.2 Class Diagram

The class diagram defines the following inheritance and association relationships:

- User is an abstract superclass. Admin, HRManager, and DeptHead extend User.
- Organization aggregates HRManager (1 to many) and JobPost (1 to many).
- JobPost aggregates CVRecord (1 to many) and Message (1 to many).
- CVRecord is associated with exactly one Resume and exactly one ScreeningResult.
- Shortlist associates HRManager with a CVRecord through a JobPost context.
- Feedback is associated with User (one to many).



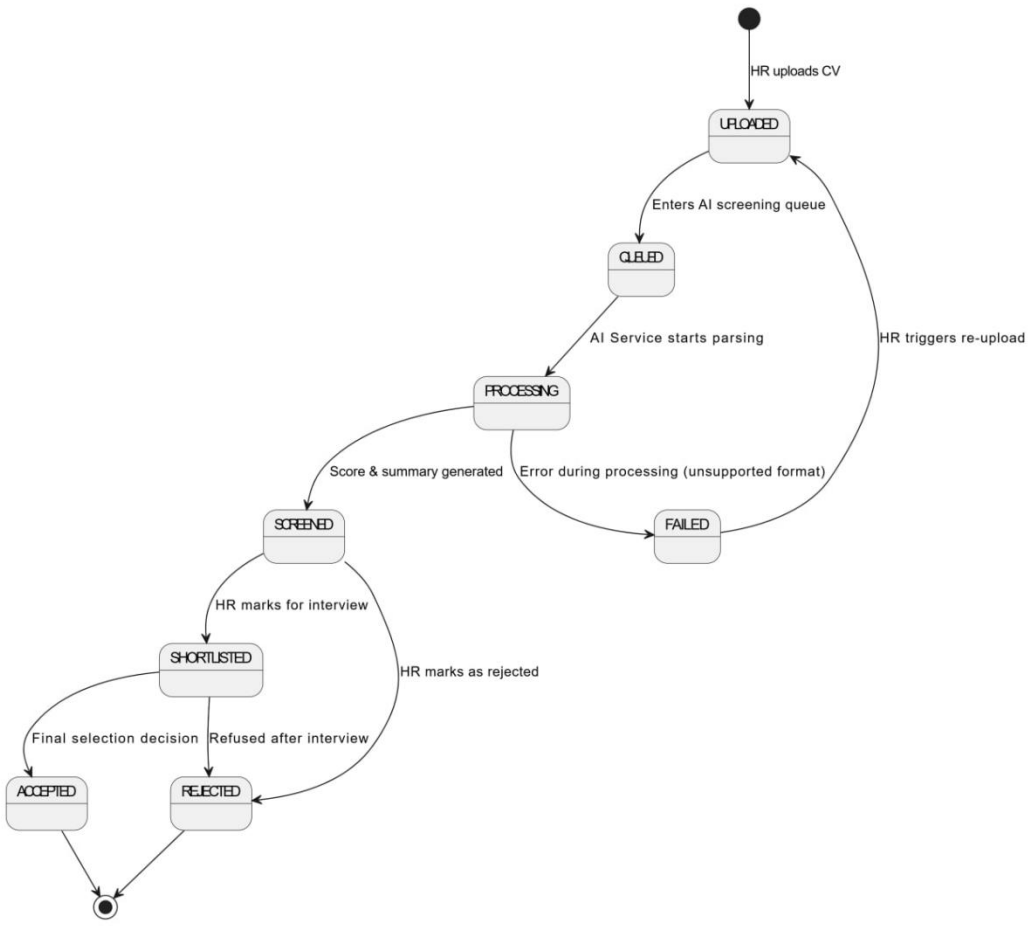
9. Behavioral Modeling:

9.1 State Transition Diagram

The following state machines define the lifecycle of the two most critical objects in the NextHire system:

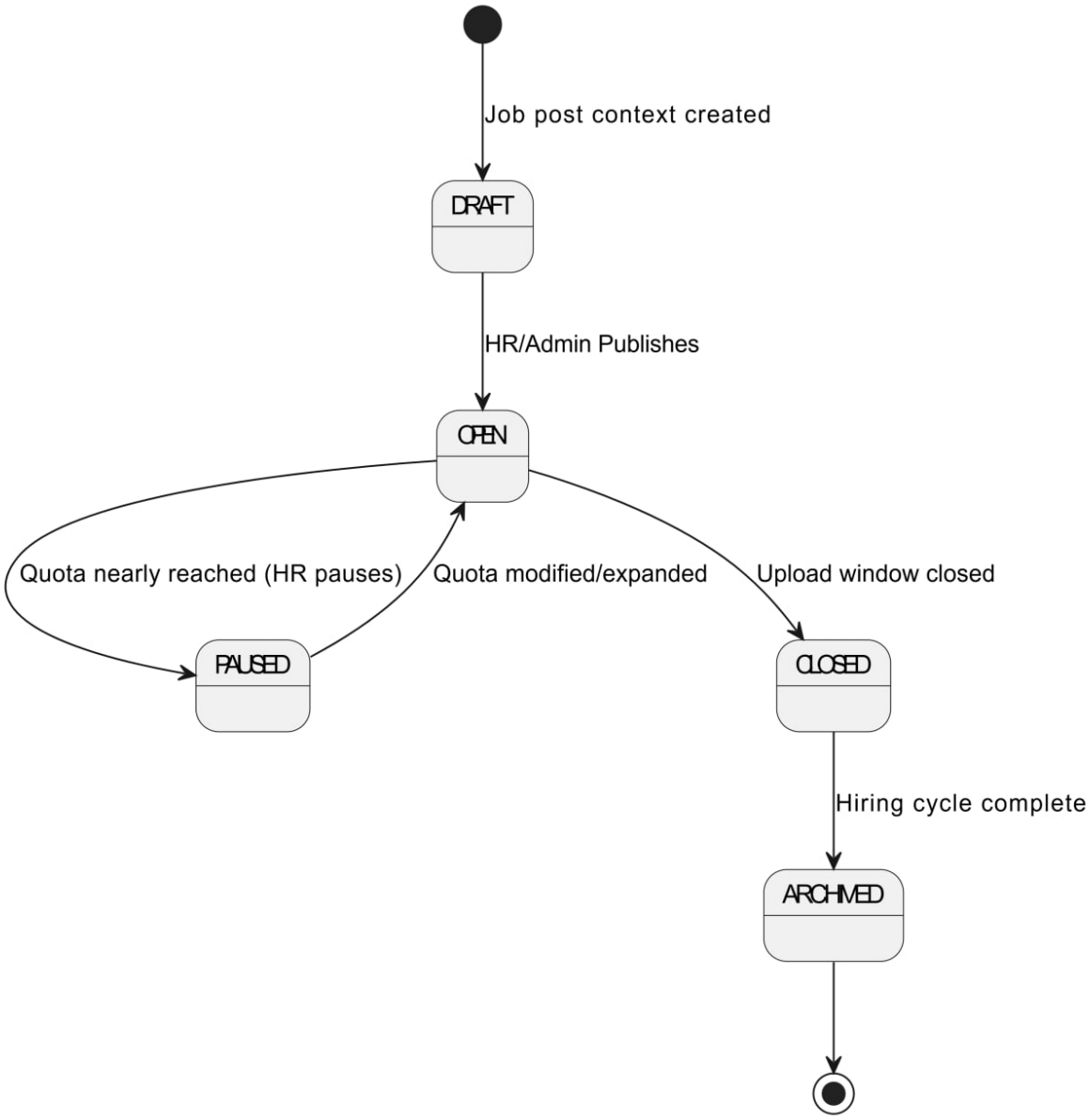
CVRecord State Machine:

- **UPLOADED**: Initial state after HR uploads a candidate's CV against a job post.
- **QUEUED**: CV enters the AI screening queue.
- **PROCESSING**: AI micro-service is actively parsing and scoring the resume.
- **SCREENED**: Score and summary are generated; candidate appears in ranked list.
- **SHORTLISTED**: HR manager marks candidate for interview.
- **ACCEPTED**: Final positive decision recorded.
- **REJECTED**: Candidate not selected; decision recorded for internal records.
- **FAILED**: CV processing failed (e.g., unsupported format); available for re-upload.



JobPost State Machine:

- **DRAFT**: Job post created but not yet active.
- **OPEN**: Post active; HR can upload CVs against it.
- **PAUSED**: HR temporarily stops accepting new CV uploads (quota nearly reached).
- **CLOSED**: Upload window closed; shortlisting in progress.
- **ARCHIVED**: Hiring cycle complete; post moved to historical records.



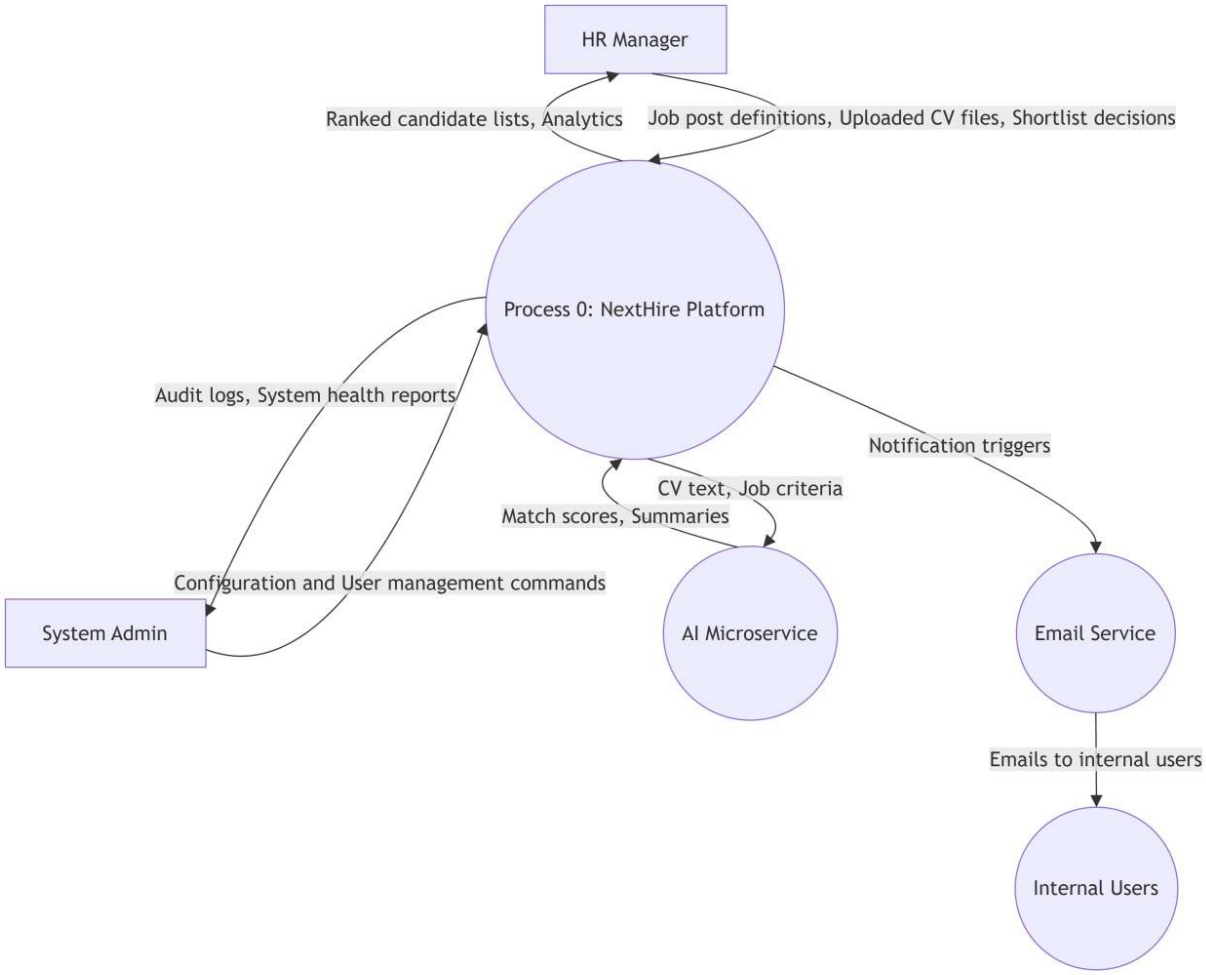
10. Data Flow Modeling:

Data Flow Diagrams (DFDs) model the movement of data through the NextHire system across multiple levels of abstraction.

10.1 Data Flow Diagram (Level 0 – Context)

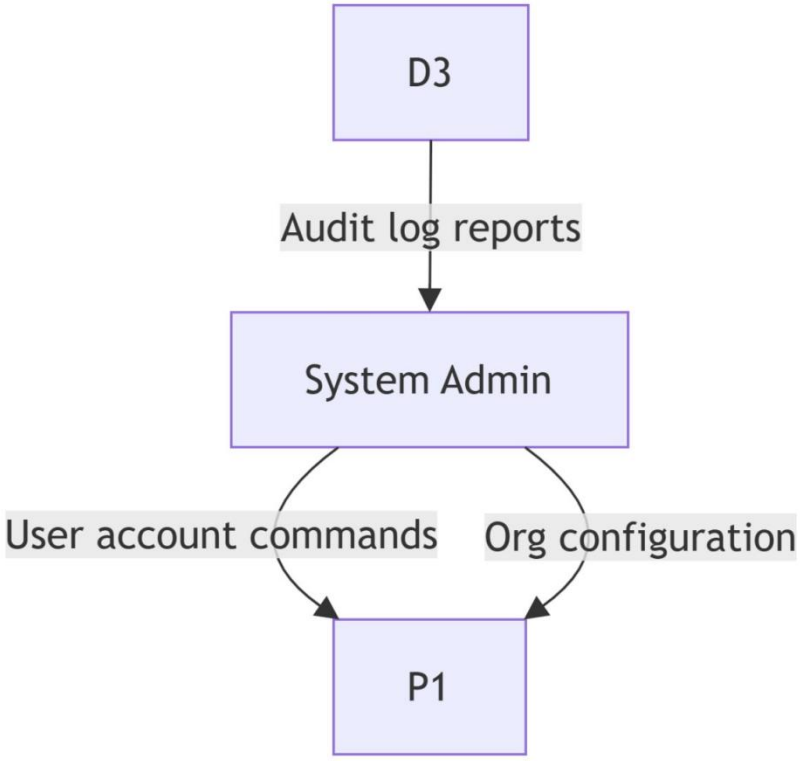
At Level 0, NextHire is represented as a single process bubble. External entities are:

- HR Manager: Sends job post definitions, uploaded CV files, and shortlist decisions in; receives ranked candidate lists and analytics out.
- System Admin: Sends configuration and user management commands in; receives audit logs and system health reports out.
- AI Microservice: Receives CV text and job criteria in; sends match scores and summaries out.
- Email Service: Receives notification triggers in; sends emails to internal users out.



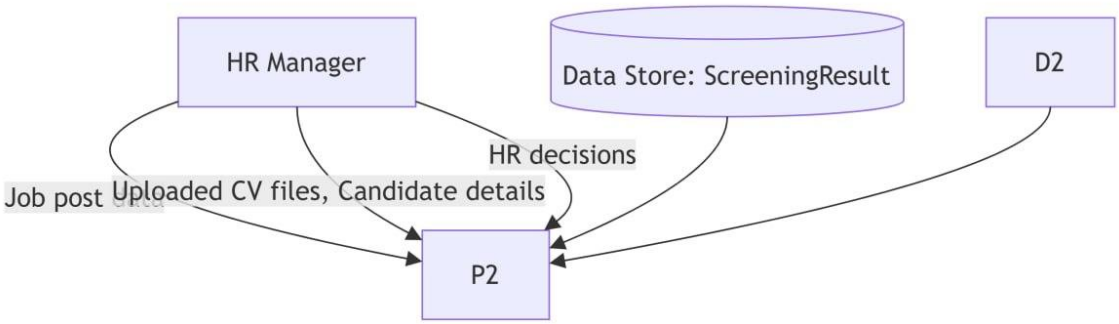
10.2 Data Flow Diagram (Level 1 – Admin)

Admin data flows: User Management Module receives user account commands from Admin → reads/writes User data store → Organization Management Module receives org configuration → reads/writes Organization data store → Audit Log Module receives all action events → writes to Audit Log data store → Admin reads audit log reports.



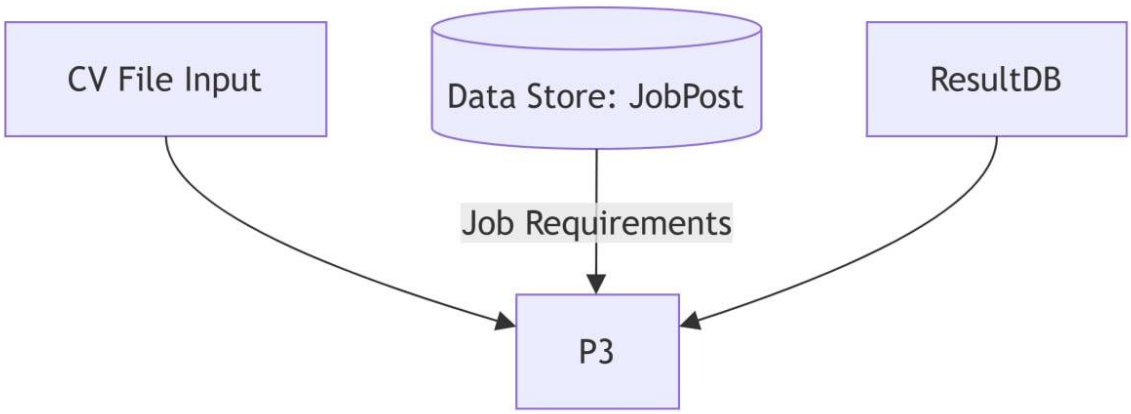
10.3 Data Flow Diagram (Level 1 – HR Manager)

HR data flows: Job Creation Module receives job post data → writes to JobPost data store → CV Upload Module receives uploaded CV files and candidate details → writes to CVRecord and Resume data stores → triggers AI Screening pipeline → Screening Dashboard reads ScreeningResult and CVRecord data stores → presents ranked list to HR → Shortlisting Module receives HR decisions → writes to Shortlist data store.



10.4 Data Flow Diagram (Level 2 – AI Screening Detail)

Level 2 decomposition of the AI Screening process: CV File → Text Extraction Process → Raw Text → NLP Parsing Process → Structured Entities (skills, education, experience) → Match Scoring Process (inputs: Structured Entities + Job Requirements from JobPost data store) → Match Score → Summary Generation Process → AI Summary → ScreeningResult data store → Ranking Process reads ScreeningResult data store → updates CVRecord data store with rank field.



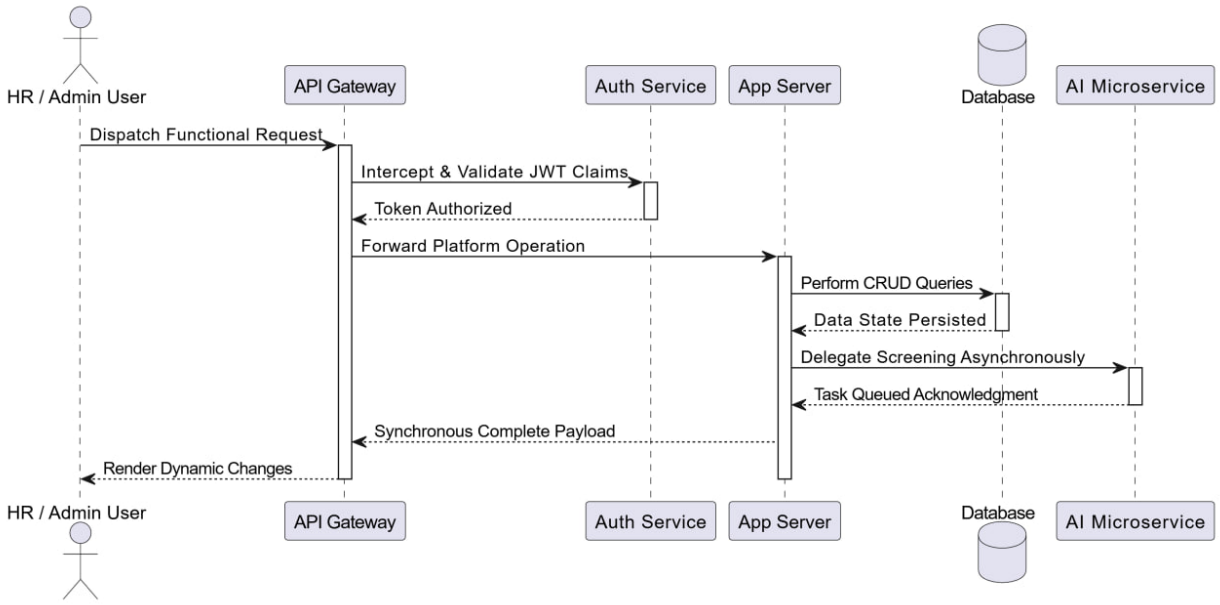
11. Sequence Diagrams

Sequence diagrams illustrate the time-ordered interactions between objects for each major system flow.

11.1 Overall System Sequence

Participants: User Browser → API Gateway → Auth Service → Application Server → Database → AI Microservice

Summary: All requests pass through the API Gateway. The Auth Service validates JWT on each request. The Application Server processes business logic, interacts with the Database for persistence, and delegates screening tasks to the AI Microservice asynchronously. The AI Microservice returns results via a callback endpoint, which updates the CVRecord and notifies the frontend via WebSocket push.



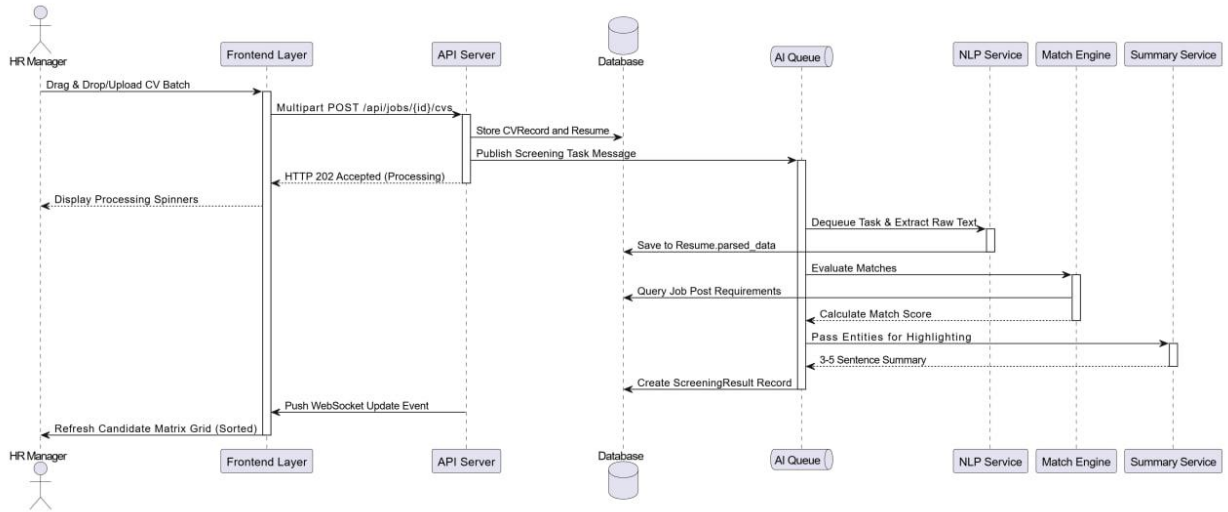
11.2 CV Upload & AI Screening

Participants: HR Manager → Frontend → API Server → Database → AI Queue → NLP Service → Match Engine → Summary Service

Sequence:

1. HR Manager uploads one or more candidate CV files against a job post.
2. Frontend sends multipart POST to /api/jobs/{id}/cvs.
3. API Server validates and stores CVRecord and Resume records in Database.

4. API Server publishes a screening job to the AI Queue for each uploaded CV.
5. NLP Service dequeues job, extracts text, parses entities, stores in Resume.parsed_data.
6. Match Engine retrieves JobPost requirements from Database, computes score.
7. Summary Service generates AI text summary.
8. Results stored in ScreeningResult record.
9. API Server pushes a ranking update notification to the HR dashboard via WebSocket.

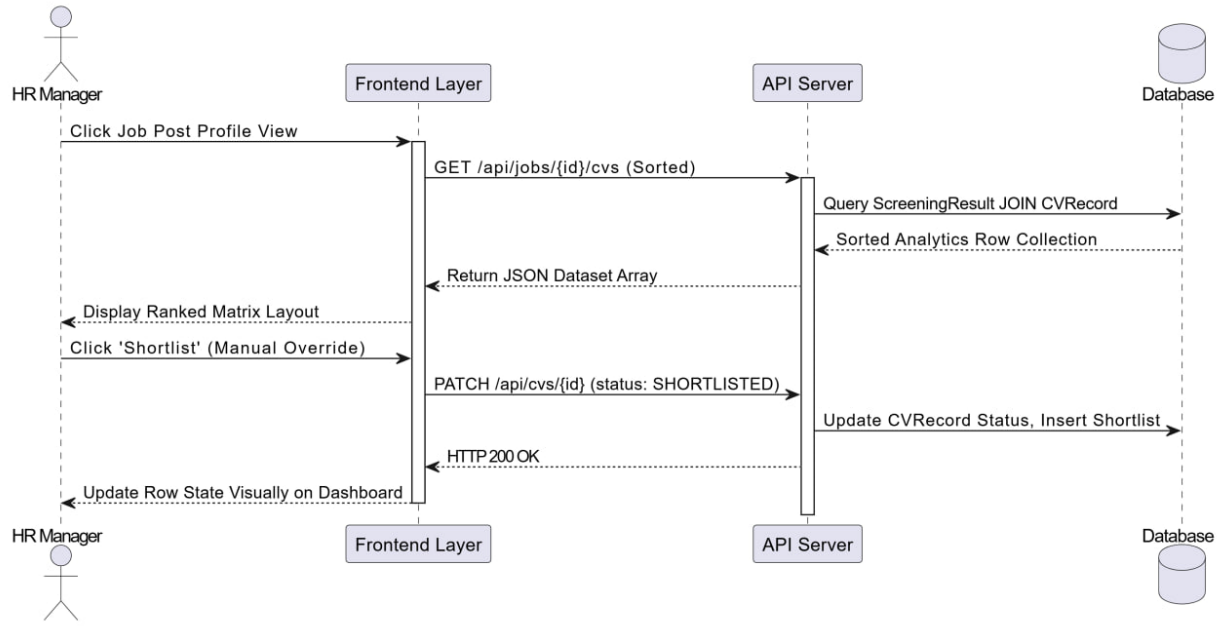


11.3 HR Shortlisting Flow

Participants: HR Manager → Frontend → API Server → Database

Sequence:

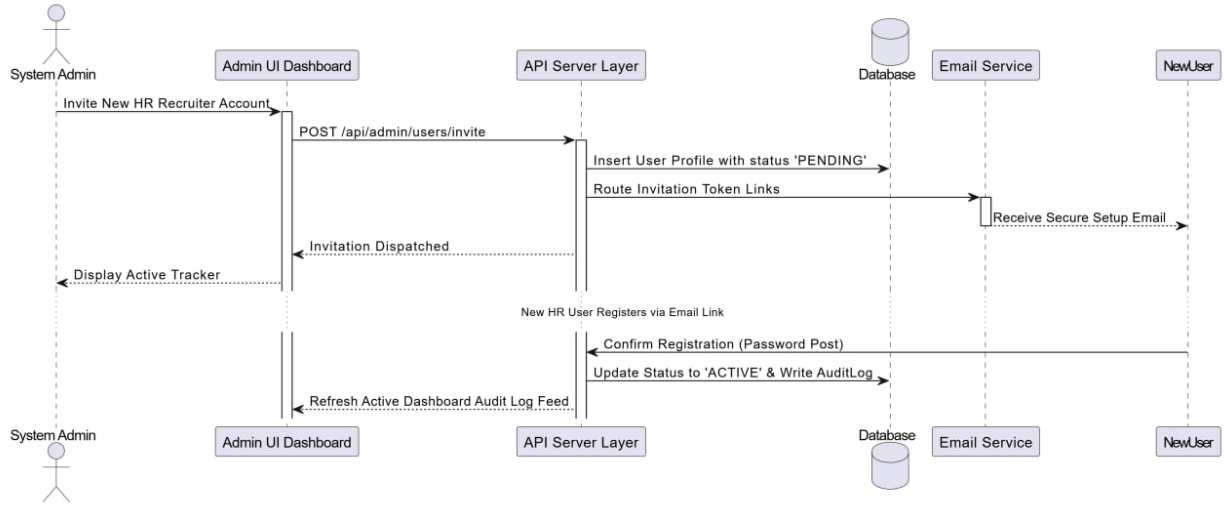
1. HR Manager opens the candidate ranking dashboard.
2. Frontend sends GET /api/jobs/{id}/cvs (sorted by match score).
3. API Server queries ScreeningResult joined with CVRecord.
4. HR reviews the list and clicks ‘Shortlist’ on selected candidates.
5. Frontend sends PATCH /api/cvs/{id} with status: SHORTLISTED.
6. API Server updates the CVRecord and creates a Shortlist record.
7. Updated status is reflected immediately on the HR dashboard.



11.4 Admin User & System Management

Participants: System Admin → Admin Dashboard → API Server → Database → Email Service

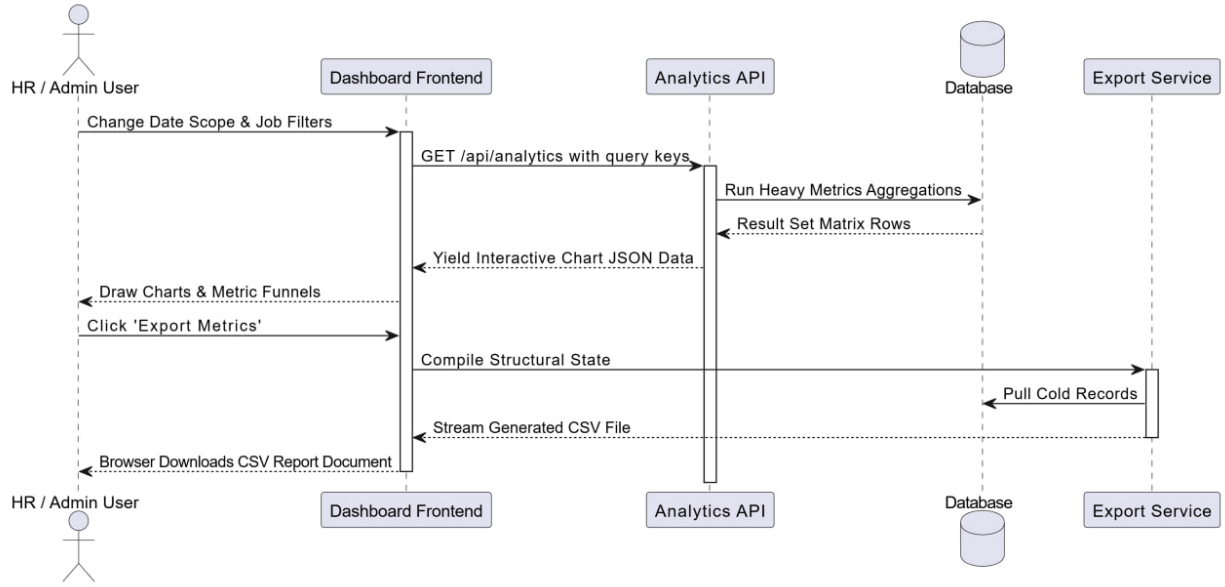
Sequence: Admin creates new HR user account → System generates temporary credentials → Email Service sends invitation → New user registers and sets password → Database updates user status to ACTIVE → Admin can view audit log of all account activity.



11.5 Analytics Report Generation

Participants: HR/Admin → Dashboard Frontend → Analytics API → Database → Export Service

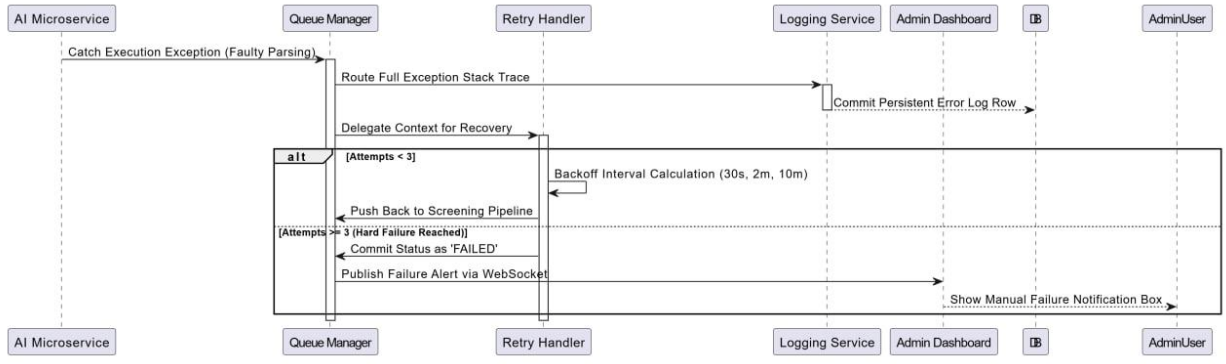
Sequence: User applies filters (date range, job post) → Frontend sends GET /api/analytics with query params → Analytics API executes aggregation queries → Database returns aggregated metrics → API formats response (charts data + table) → Frontend renders visualizations → User optionally clicks Export → Export Service generates CSV → Browser downloads file.



11.6 Error Recovery Sequence

Participants: AI Microservice → Queue Manager → Retry Handler → Logging Service → Admin Dashboard

Sequence: AI Microservice fails during processing → Returns error to Queue Manager → Queue Manager logs failure to Logging Service → Retry Handler re-queues job with exponential backoff (retry 1: 30s, retry 2: 2min, retry 3: 10min) → After 3 failures, marks job as FAILED → Admin Dashboard receives alert → Admin investigates Logging Service output → Manual retry available from Admin panel.



Appendix

A. Glossary

Term	Definition
NLP	Natural Language Processing AI techniques for parsing and understanding human text
JWT	JSON Web Token a compact, URL-safe format for securely transmitting authentication claims
Match Score	A percentage (0–100) representing how well a candidate's CV aligns with a job post's requirements
Microservice	An independently deployable service responsible for a single bounded function (here, the AI pipeline)
Shortlist	The subset of uploaded CVs selected by HR for the interview stage
Tenant	A single organization with its own isolated data space within the multi-tenant NextHire platform
DXA	Twentieths of a point a unit of measurement used in OOXML document formatting
REST API	Representational State Transfer Application Programming Interface the web communication standard used by NextHire
ERD	Entity Relationship Diagram is a visual model of database entities and their relationships
SRS	Software Requirements Specification this document

B. References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Pressman, R.S. (2014). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- HuggingFace Transformers Documentation: https://huggingface.co/docs/transformers
- spaCy NLP Library Documentation: https://spacy.io/usage
- NextHire Project Proposal — Team Innovex, May 2026